

**Employee Performance Mapping
Course-End Project**

**Submitted By:
Siddhant Waghmare**

**Course:
Data Acquisition and Manipulation using SQL**

**Tool Used:
MySQL Workbench**

Table of Contents

1. Introduction
2. Objectives of the Project
3. Database Creation
4. Table Creation
5. Data Import Process
6. ER Diagram
7. SQL Queries and Outputs
 - 7.1 Employee Department Details
 - 7.2 Employee Rating Analysis
 - 7.3 Finance Department Employee Names
 - 7.4 Employees Having Reporters
 - 7.5 Healthcare and Finance Employees using UNION
 - 7.6 Department-wise Maximum Employee Rating
 - 7.7 Minimum and Maximum Salary by Role
 - 7.8 Employee Ranking based on Experience
 - 7.9 View Creation for High Salary Employees
 - 7.10 Nested Query for Experienced Employees
 - 7.11 Stored Procedure
 - 7.12 Stored Function
 - 7.13 Index Creation and Execution Plan
 - 7.14 Bonus Calculation
 - 7.15 Average Salary Distribution
8. Conclusion

1. Introduction

The Employee Performance Mapping project was developed to analyse employee data for ScienceQtech, a startup operating in the Data Science domain. The project focuses on employee performance evaluation, salary analysis, departmental analysis, reporting hierarchy, and project allocation using SQL.

The project was implemented using MySQL Workbench. Various SQL concepts such as DDL commands, DML commands, aggregate functions, joins, subqueries, views, stored procedures, stored functions, indexes, and window functions were used to perform data analysis and management operations effectively.

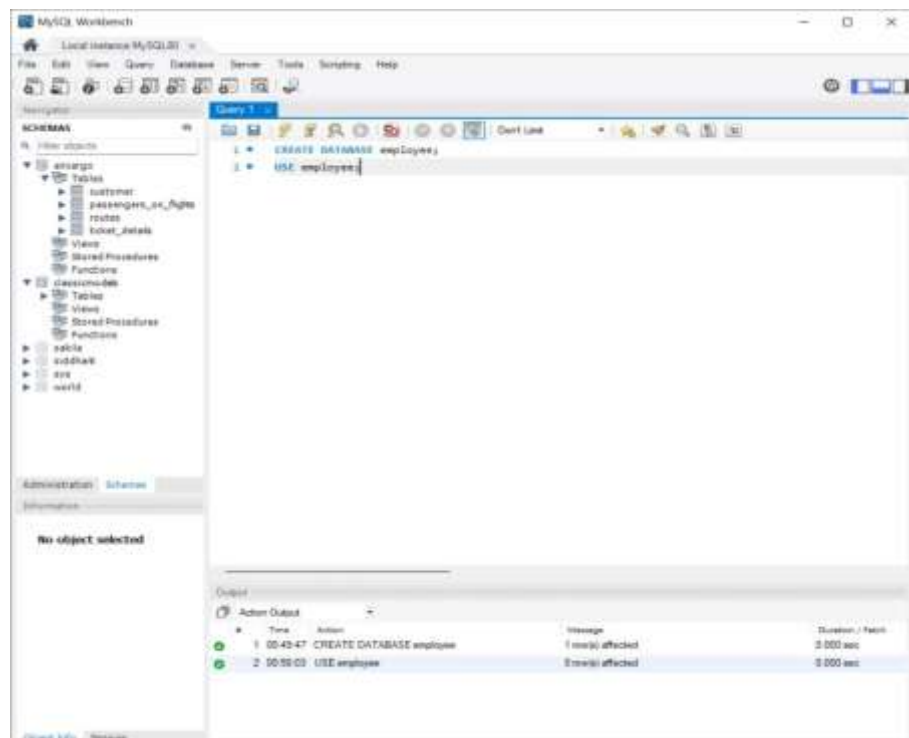
2. Objectives of the Project

- To create and manage employee databases using SQL.
- To analyse employee performance ratings.
- To evaluate salary distribution across departments and countries.
- To identify reporting hierarchy among employees.
- To implement views, stored procedures, and functions.
- To improve query performance using indexing.
- To perform employee performance mapping for HR analysis.

3. Database Creation

CREATE DATABASE employee;

USE employee;



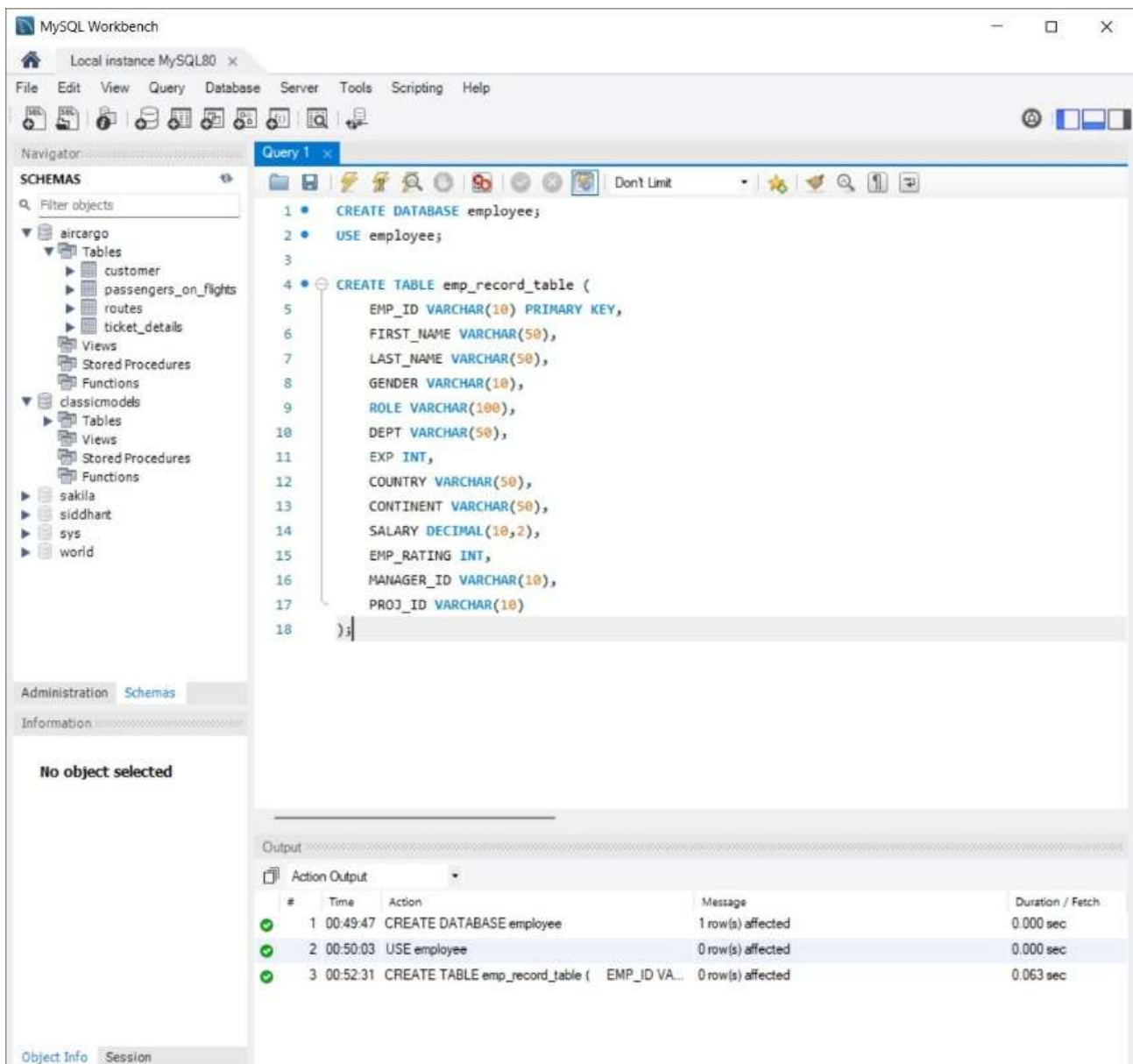
4. Table Creation

```
CREATE TABLE emp_record_table (  
    EMP_ID VARCHAR(10) PRIMARY KEY,  
    FIRST_NAME VARCHAR(50),  
    LAST_NAME VARCHAR(50),  
    GENDER VARCHAR(10),  
    ROLE VARCHAR(100),  
    DEPT VARCHAR(50),  
    EXP INT,  
    COUNTRY VARCHAR(50),  
    CONTINENT VARCHAR(50),  
    SALARY DECIMAL(10,2),  
    EMP_RATING INT,  
    MANAGER_ID VARCHAR(10),  
    PROJ_ID VARCHAR(10)  
);
```

```
CREATE TABLE proj_table (  
    PROJECT_ID VARCHAR(10) PRIMARY KEY,  
    PROJ_NAME VARCHAR(100),  
    DOMAIN VARCHAR(100),  
    START_DATE DATE,  
    CLOSURE_DATE DATE,  
    DEV_QTR VARCHAR(10),  
    STATUS VARCHAR(50)  
);
```

```
CREATE TABLE data_science_team (  
    EMP_ID VARCHAR(10) PRIMARY KEY,
```

```
FIRST_NAME VARCHAR(50),  
LAST_NAME VARCHAR(50),  
GENDER VARCHAR(10),  
ROLE VARCHAR(100),  
DEPT VARCHAR(50),  
EXP INT,  
COUNTRY VARCHAR(50),  
CONTINENT VARCHAR(50)  
);
```



MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- aircargo
 - Tables
 - customer
 - passengers_on_flights
 - routes
 - ticket_details
 - Views
 - Stored Procedures
 - Functions
- classmodels
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sakila
- siddhart
- sys
- world

Administration Schemas

Information

No object selected

Query 1

```

5  EMP_ID VARCHAR(10) PRIMARY KEY,
6  FIRST_NAME VARCHAR(50),
7  LAST_NAME VARCHAR(50),
8  GENDER VARCHAR(10),
9  ROLE VARCHAR(100),
10 DEPT VARCHAR(50),
11 EXP INT,
12 COUNTRY VARCHAR(50),
13 CONTINENT VARCHAR(50),
14 SALARY DECIMAL(10,2),
15 EMP_RATING INT,
16 MANAGER_ID VARCHAR(10),
17 PROJ_ID VARCHAR(10)
18 })
19
20 CREATE TABLE proj_table (
21   PROJECT_ID VARCHAR(10) PRIMARY KEY,
22   PROJ_NAME VARCHAR(100),
23   DOMAIN VARCHAR(100),
24   START_DATE DATE,
25   CLOSURE_DATE DATE,
26   DEV_QTR VARCHAR(10),
27   STATUS VARCHAR(50)
28 );

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	00:49:47	CREATE DATABASE employee	1 row(s) affected	0.000 sec
2	00:50:03	USE employee	0 row(s) affected	0.000 sec
3	00:52:31	CREATE TABLE emp_record_table (EMP_ID VA...	0 row(s) affected	0.063 sec
4	00:55:51	CREATE TABLE proj_table (PROJECT_ID VARC...	0 row(s) affected	0.047 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- aircargo
 - Tables
 - customer
 - passengers_on_flights
 - routes
 - ticket_details
 - Views
 - Stored Procedures
 - Functions
- classmodels
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sakila
- siddhart
- sys
- world

Administration Schemas

Information

No object selected

Query 1

```

17   PROJ_ID VARCHAR(10)
18 })
19
20 CREATE TABLE proj_table (
21   PROJECT_ID VARCHAR(10) PRIMARY KEY,
22   PROJ_NAME VARCHAR(100),
23   DOMAIN VARCHAR(100),
24   START_DATE DATE,
25   CLOSURE_DATE DATE,
26   DEV_QTR VARCHAR(10),
27   STATUS VARCHAR(50)
28 );
29
30 CREATE TABLE data_science_team (
31   EMP_ID VARCHAR(10) PRIMARY KEY,
32   FIRST_NAME VARCHAR(50),
33   LAST_NAME VARCHAR(50),
34   GENDER VARCHAR(10),
35   ROLE VARCHAR(100),
36   DEPT VARCHAR(50),
37   EXP INT,
38   COUNTRY VARCHAR(50),
39   CONTINENT VARCHAR(50)
40 );

```

Output

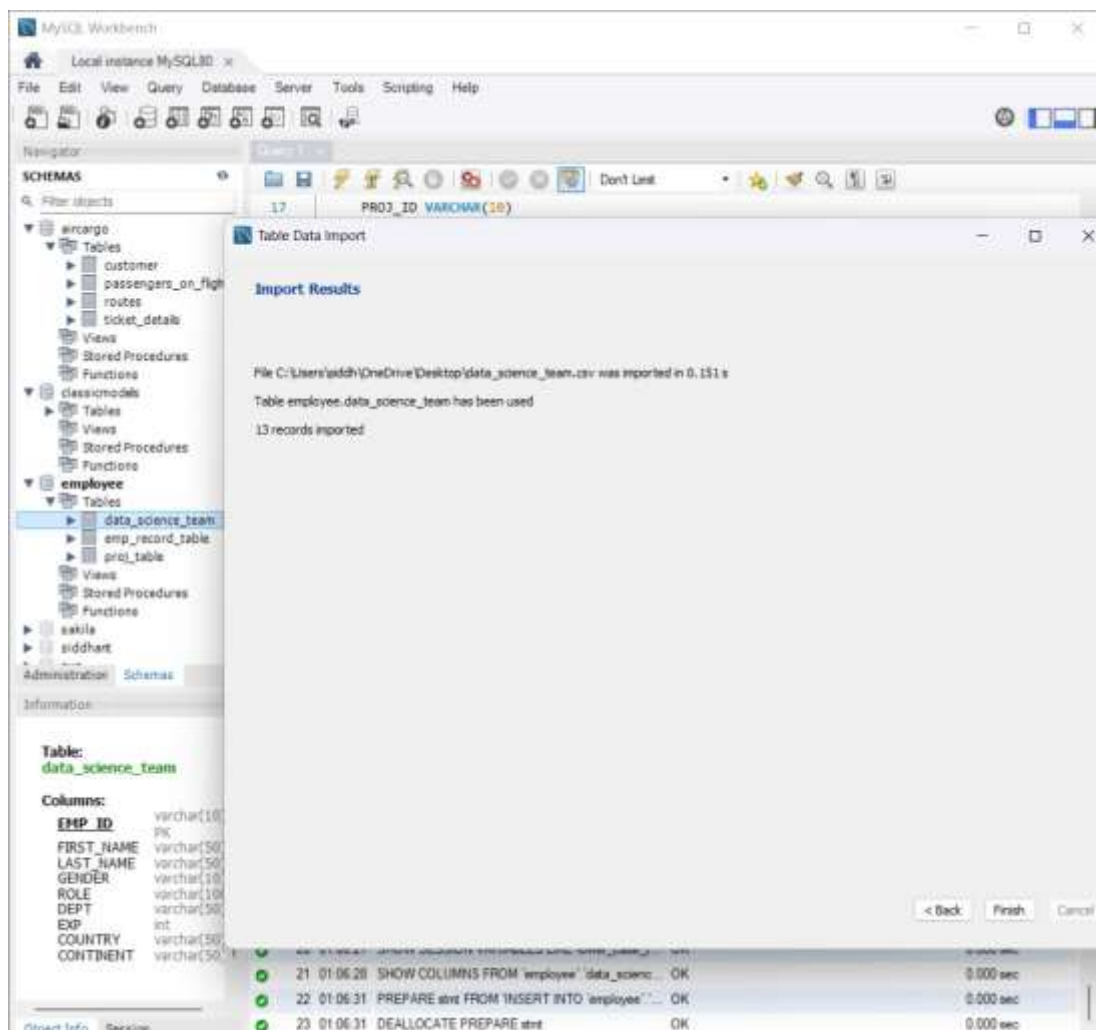
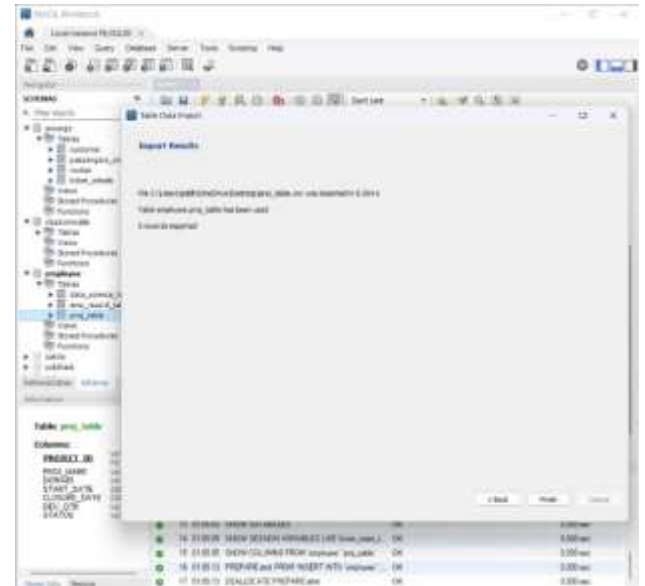
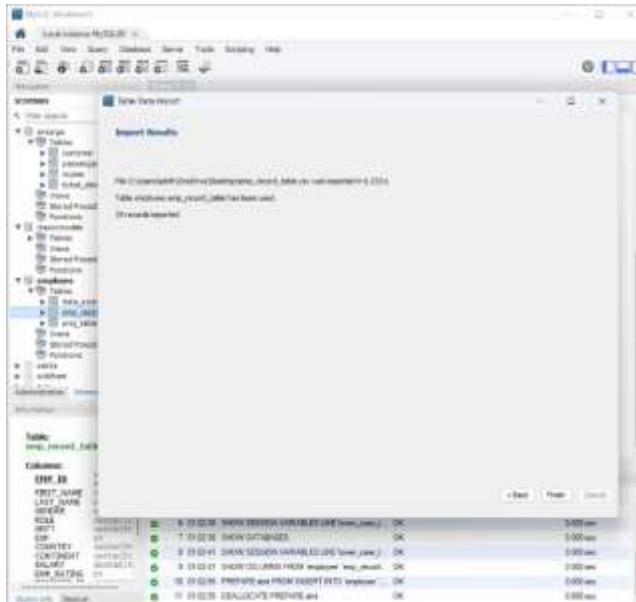
Action Output

#	Time	Action	Message	Duration / Fetch
1	00:49:47	CREATE DATABASE employee	1 row(s) affected	0.000 sec
2	00:50:03	USE employee	0 row(s) affected	0.000 sec
3	00:52:31	CREATE TABLE emp_record_table (EMP_ID VA...	0 row(s) affected	0.063 sec
4	00:55:51	CREATE TABLE proj_table (PROJECT_ID VARC...	0 row(s) affected	0.047 sec
5	00:57:00	CREATE TABLE data_science_team (EMP_ID V...	0 row(s) affected	0.062 sec

Object Info Session

5. Data Import Process

The CSV files were imported into MySQL Workbench using the Table Data Import Wizard. The datasets included employee records, project details, and data science team information.



MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigation

SCHEMAS

Filter objects

airways

- customer
- passengers_on_flight
- routes
- ticket_details
- views
- Stored Procedures
- Functions

classmate_db

- Tables
- Views
- Stored Procedures
- Functions

employee

- Tables
- data_science_team
- emp_record_table
- emp_table
- views
- Stored Procedures
- Functions

Schema

Information

Table: data_science_team

Columns:

- EMP_ID: varchar(10)
- FIRST_NAME: varchar(50)
- LAST_NAME: varchar(50)
- GENDER: varchar(10)
- ROLE: varchar(10)
- DEPT: varchar(20)
- EXP: int
- COUNTRY: varchar(20)
- CONTINENT: varchar(50)

Query 1

```

12 FIRST_NAME VARCHAR(50),
13 LAST_NAME VARCHAR(50),
14 GENDER VARCHAR(10),
15 ROLE VARCHAR(10),
16 DEPT VARCHAR(20),
17 EXP INT,
18 COUNTRY VARCHAR(20),
19 CONTINENT VARCHAR(50)
40 })
41
42 * SELECT * FROM emp_record_table;

```

Result grid

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	ROLE	DEPT	EXP	COUNTRY	CONTINENT	SALARY	EMP_RATING	MANAGER_ID	PROJ_ID
1001	Arthur	Bash	M	PRESIDENT	ALL	20	USA	NORTH AMERICA	2500.00	5	000	000
1005	Bob	Huffman	M	LEAD DATA SCIENTIST	FINANCE	13	USA	NORTH AMERICA	8500.00	3	0103	P105
1010	Indian	Buller	M	LEAD DATA SCIENTIST	AUTOCOTIVE	12	FRANCE	EUROPE	8000.00	2	0108	P104
1002	Diana	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	8	CANADA	NORTH AMERICA	9500.00	8	0102	P103
1007	Dorothy	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	9	USA	NORTH AMERICA	7500.00	1	0101	P102
1003	Patrick	Walt	M	MANAGER	HEALTHCARE	18	USA	NORTH AMERICA	9500.00	5	0101	000
1004	Emily	Grove	F	MANAGER	FINANCE	14	CANADA	NORTH AMERICA	2500.00	4	0101	000
1004	Rayne	Novak	F	SENIOR DATA SCIENTIST	AUTOCOTIVE	8	GERMANY	EUROPE	7000.00	5	0108	P104
1016	Neil	Zhou	M	SENIOR DATA SCIENTIST	SETAS	6	CHINA	ASIA	4500.00	2	0103	P104
1000	Ray	Collier	M	SENIOR DATA SCIENTIST	SETAS	7	INDIA	ASIA	2000.00	3	0103	NA
1003	Steve	Huffman	M	ASSOCIATE DATA SCIENTIST	FINANCE	4	USA	NORTH AMERICA	8000.00	3	0103	P103
1020	Paul	Allen	M	MANAGER	AUTOCOTIVE	14	GERMANY	EUROPE	11000.00	4	0101	000
1018	Daniel	Smith	M	ASSOCIATE DATA SCIENTIST	SETAS	3	COLOMBIA	SOUTH AMERICA	4000.00	4	0102	P109

emp_record_table

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	ROLE	DEPT	EXP	COUNTRY	CONTINENT	SALARY	EMP_RATING	MANAGER_ID	PROJ_ID
1001	Arthur	Bash	M	PRESIDENT	ALL	20	USA	NORTH AMERICA	2500.00	5	000	000
1005	Bob	Huffman	M	LEAD DATA SCIENTIST	FINANCE	13	USA	NORTH AMERICA	8500.00	3	0103	P105
1010	Indian	Buller	M	LEAD DATA SCIENTIST	AUTOCOTIVE	12	FRANCE	EUROPE	8000.00	2	0108	P104
1002	Diana	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	8	CANADA	NORTH AMERICA	9500.00	8	0102	P103
1007	Dorothy	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	9	USA	NORTH AMERICA	7500.00	1	0101	P102
1003	Patrick	Walt	M	MANAGER	HEALTHCARE	18	USA	NORTH AMERICA	9500.00	5	0101	000
1004	Emily	Grove	F	MANAGER	FINANCE	14	CANADA	NORTH AMERICA	2500.00	4	0101	000
1004	Rayne	Novak	F	SENIOR DATA SCIENTIST	AUTOCOTIVE	8	GERMANY	EUROPE	7000.00	5	0108	P104
1016	Neil	Zhou	M	SENIOR DATA SCIENTIST	SETAS	6	CHINA	ASIA	4500.00	2	0103	P104
1000	Ray	Collier	M	SENIOR DATA SCIENTIST	SETAS	7	INDIA	ASIA	2000.00	3	0103	NA
1003	Steve	Huffman	M	ASSOCIATE DATA SCIENTIST	FINANCE	4	USA	NORTH AMERICA	8000.00	3	0103	P103
1020	Paul	Allen	M	MANAGER	AUTOCOTIVE	14	GERMANY	EUROPE	11000.00	4	0101	000
1018	Daniel	Smith	M	ASSOCIATE DATA SCIENTIST	SETAS	3	COLOMBIA	SOUTH AMERICA	4000.00	4	0102	P109

emp_table

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	ROLE	DEPT	EXP	COUNTRY	CONTINENT	SALARY	EMP_RATING	MANAGER_ID	PROJ_ID
1001	Arthur	Bash	M	PRESIDENT	ALL	20	USA	NORTH AMERICA	2500.00	5	000	000
1005	Bob	Huffman	M	LEAD DATA SCIENTIST	FINANCE	13	USA	NORTH AMERICA	8500.00	3	0103	P105
1010	Indian	Buller	M	LEAD DATA SCIENTIST	AUTOCOTIVE	12	FRANCE	EUROPE	8000.00	2	0108	P104
1002	Diana	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	8	CANADA	NORTH AMERICA	9500.00	8	0102	P103
1007	Dorothy	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	9	USA	NORTH AMERICA	7500.00	1	0101	P102
1003	Patrick	Walt	M	MANAGER	HEALTHCARE	18	USA	NORTH AMERICA	9500.00	5	0101	000
1004	Emily	Grove	F	MANAGER	FINANCE	14	CANADA	NORTH AMERICA	2500.00	4	0101	000
1004	Rayne	Novak	F	SENIOR DATA SCIENTIST	AUTOCOTIVE	8	GERMANY	EUROPE	7000.00	5	0108	P104
1016	Neil	Zhou	M	SENIOR DATA SCIENTIST	SETAS	6	CHINA	ASIA	4500.00	2	0103	P104
1000	Ray	Collier	M	SENIOR DATA SCIENTIST	SETAS	7	INDIA	ASIA	2000.00	3	0103	NA
1003	Steve	Huffman	M	ASSOCIATE DATA SCIENTIST	FINANCE	4	USA	NORTH AMERICA	8000.00	3	0103	P103
1020	Paul	Allen	M	MANAGER	AUTOCOTIVE	14	GERMANY	EUROPE	11000.00	4	0101	000
1018	Daniel	Smith	M	ASSOCIATE DATA SCIENTIST	SETAS	3	COLOMBIA	SOUTH AMERICA	4000.00	4	0102	P109

Output

Action Output

Time	Action	Message	Duration / Feeds
19 01:06:25	SHOW DATABASES	OK	0.000 sec
20 01:06:27	SHOW SESSION VARIABLES LIKE lower_case_table_names	OK	0.000 sec
21 01:06:28	SHOW COLUMNS FROM employee`data_science_team`	OK	0.000 sec
22 01:06:31	PREPARE stmt FROM INSERT INTO employee`data_science_team` (EMP_ID, FIRST_NAME, LAST_NA...	OK	0.000 sec
23 01:06:31	DEALLOCATE PREPARE stmt	OK	0.000 sec
24 01:07:59	SELECT * FROM emp_record_table	13 rows returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigation

SCHEMAS

Filter objects

airways

- customer
- passengers_on_flight
- routes
- ticket_details
- views
- Stored Procedures
- Functions

classmate_db

- Tables
- Views
- Stored Procedures
- Functions

employee

- Tables
- data_science_team
- emp_record_table
- emp_table
- views
- Stored Procedures
- Functions

Schema

Information

Table: data_science_team

Columns:

- EMP_ID: varchar(10)
- FIRST_NAME: varchar(50)
- LAST_NAME: varchar(50)
- GENDER: varchar(10)
- ROLE: varchar(10)
- DEPT: varchar(20)
- EXP: int
- COUNTRY: varchar(20)
- CONTINENT: varchar(50)

Query 1

```

12 LAST_NAME VARCHAR(50),
13 GENDER VARCHAR(10),
14 ROLE VARCHAR(10),
15 DEPT VARCHAR(20),
16 EXP INT,
17 COUNTRY VARCHAR(20),
18 CONTINENT VARCHAR(50)
40 })
41
42 * SELECT * FROM emp_record_table;
43 * SELECT * FROM emp_table;

```

Result grid

PROJECT_ID	PROJ_NAME	DOMAIN	START_DATE	CLOSURE_DATE	DEV_QTR	STATUS
000						

emp_table

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	ROLE	DEPT	EXP	COUNTRY	CONTINENT	SALARY	EMP_RATING	MANAGER_ID	PROJ_ID
1001	Arthur	Bash	M	PRESIDENT	ALL	20	USA	NORTH AMERICA	2500.00	5	000	000
1005	Bob	Huffman	M	LEAD DATA SCIENTIST	FINANCE	13	USA	NORTH AMERICA	8500.00	3	0103	P105
1010	Indian	Buller	M	LEAD DATA SCIENTIST	AUTOCOTIVE	12	FRANCE	EUROPE	8000.00	2	0108	P104
1002	Diana	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	8	CANADA	NORTH AMERICA	9500.00	8	0102	P103
1007	Dorothy	Wilson	F	SENIOR DATA SCIENTIST	HEALTHCARE	9	USA	NORTH AMERICA	7500.00	1	0101	P102
1003	Patrick	Walt	M	MANAGER	HEALTHCARE	18	USA	NORTH AMERICA	9500.00	5	0101	000
1004	Emily	Grove	F	MANAGER	FINANCE	14	CANADA	NORTH AMERICA	2500.00	4	0101	000
1004	Rayne	Novak	F	SENIOR DATA SCIENTIST	AUTOCOTIVE	8	GERMANY	EUROPE	7000.00	5	0108	P104
1016	Neil	Zhou	M	SENIOR DATA SCIENTIST	SETAS	6	CHINA	ASIA	4500.00	2	0103	P104
1000	Ray	Collier	M	SENIOR DATA SCIENTIST	SETAS	7	INDIA	ASIA	2000.00	3	0103	NA
1003	Steve	Huffman	M	ASSOCIATE DATA SCIENTIST	FINANCE	4	USA	NORTH AMERICA	8000.00	3	0103	P103
1020	Paul	Allen	M	MANAGER	AUTOCOTIVE	14	GERMANY	EUROPE	11000.00	4	0101	000
1018	Daniel	Smith	M	ASSOCIATE DATA SCIENTIST	SETAS	3	COLOMBIA	SOUTH AMERICA	4000.00	4	0102	P109

Output

Action Output

Time	Action	Message	Duration / Feeds
20 01:06:27	SHOW SESSION VARIABLES LIKE lower_case_table_names	OK	0.000 sec
21 01:06:28	SHOW COLUMNS FROM employee`data_science_team`	OK	0.000 sec
22 01:06:31	PREPARE stmt FROM INSERT INTO employee`data_science_team` (EMP_ID, FIRST_NAME, LAST_NA...	OK	0.000 sec
23 01:06:31	DEALLOCATE PREPARE stmt	OK	0.000 sec
24 01:07:59	SELECT * FROM emp_record_table	13 rows returned	0.000 sec / 0.000 sec
25 01:09:36	SELECT * FROM emp_table	0 rows returned	0.000 sec / 0.000 sec

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of databases, including 'employee'. The 'Query' pane in the center contains the following SQL code:

```

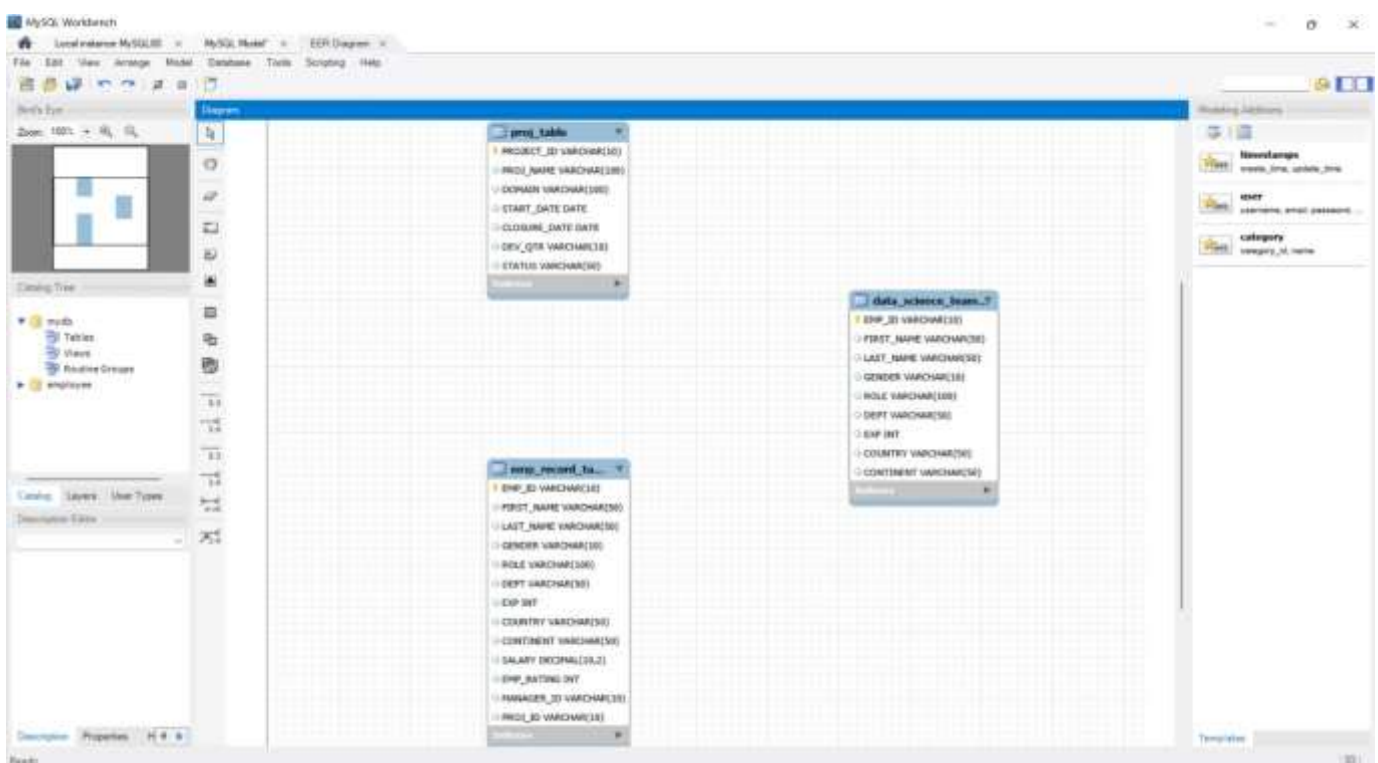
24  CREATE TABLE IF NOT EXISTS `data_science_team` (
25  `EMP_ID` VARCHAR(10),
26  `FIRST_NAME` VARCHAR(30),
27  `LAST_NAME` VARCHAR(30),
28  `GENDER` VARCHAR(10),
29  `ROLE` VARCHAR(100),
30  `DEPT` VARCHAR(50),
31  `EMP` INT,
32  `COUNTRY` VARCHAR(50),
33  `CONTINENT` VARCHAR(50)
34  ) ENGINE=InnoDB;
35
36  INSERT INTO `data_science_team` (`EMP_ID`, `FIRST_NAME`, `LAST_NAME`, `GENDER`, `ROLE`, `DEPT`, `EMP`, `COUNTRY`, `CONTINENT`) VALUES
37  ('0003', 'Eric', 'Hoffman', 'M', 'LEAD DATA SCIENTIST', 'FINANCE', '11', 'USA', 'NORTH AMERICA'),
38  ('0003', 'William', 'Buller', 'M', 'LEAD DATA SCIENTIST', 'AUTOMOTIVE', '12', 'FRANCE', 'EUROPE'),
39  ('0002', 'Danna', 'Wilson', 'F', 'SENIOR DATA SCIENTIST', 'HEALTHCARE', '9', 'CANADA', 'NORTH AMERICA'),
40  ('0007', 'Dorothea', 'Wilson', 'F', 'SENIOR DATA SCIENTIST', 'HEALTHCARE', '9', 'USA', 'NORTH AMERICA'),
41  ('0004', 'Kareem', 'Howard', 'M', 'SENIOR DATA SCIENTIST', 'AUTOMOTIVE', '8', 'GERMANY', 'EUROPE'),
42  ('0005', 'Hao', 'Zheng', 'M', 'SENIOR DATA SCIENTIST', 'RETAIL', '6', 'CHINA', 'ASIA'),
43  ('0006', 'Ray', 'Cunningham', 'M', 'SENIOR DATA SCIENTIST', 'RETAIL', '7', 'INDIA', 'ASIA'),
44  ('0001', 'Diane', 'Hoffman', 'M', 'ASSOCIATE DATA SCIENTIST', 'FINANCE', '4', 'USA', 'NORTH AMERICA'),
45  ('0008', 'David', 'Smith', 'M', 'ASSOCIATE DATA SCIENTIST', 'RETAIL', '3', 'COLOMBIA', 'SOUTH AMERICA'),
46  ('0009', 'Chad', 'Wilson', 'M', 'ASSOCIATE DATA SCIENTIST', 'HEALTHCARE', '5', 'CANADA', 'NORTH AMERICA'),
47  ('0010', 'Claire', 'Brennan', 'F', 'ASSOCIATE DATA SCIENTIST', 'AUTOMOTIVE', '3', 'GERMANY', 'EUROPE'),
48  ('0011', 'Kathleen', 'Allen', 'F', 'JUNIOR DATA SCIENTIST', 'RETAIL', '2', 'INDIA', 'ASIA'),
49  ('0012', 'Jennifer', 'Jones', 'F', 'JUNIOR DATA SCIENTIST', 'RETAIL', '1', 'COLOMBIA', 'SOUTH AMERICA');

```

Below the query, the 'Results' pane shows a table with 9 columns: EMP_ID, FIRST_NAME, LAST_NAME, GENDER, ROLE, DEPT, EMP, COUNTRY, and CONTINENT. The table contains 12 rows of data. The 'Messages' pane at the bottom shows the execution progress of the query.

6. ER Diagram

The ER Diagram represents the relationships between employee records and project details. The emp_record_table contains project IDs that are associated with the proj_table. The manager hierarchy is represented through MANAGER_ID referencing EMP_ID.



7. SQL Queries and Outputs

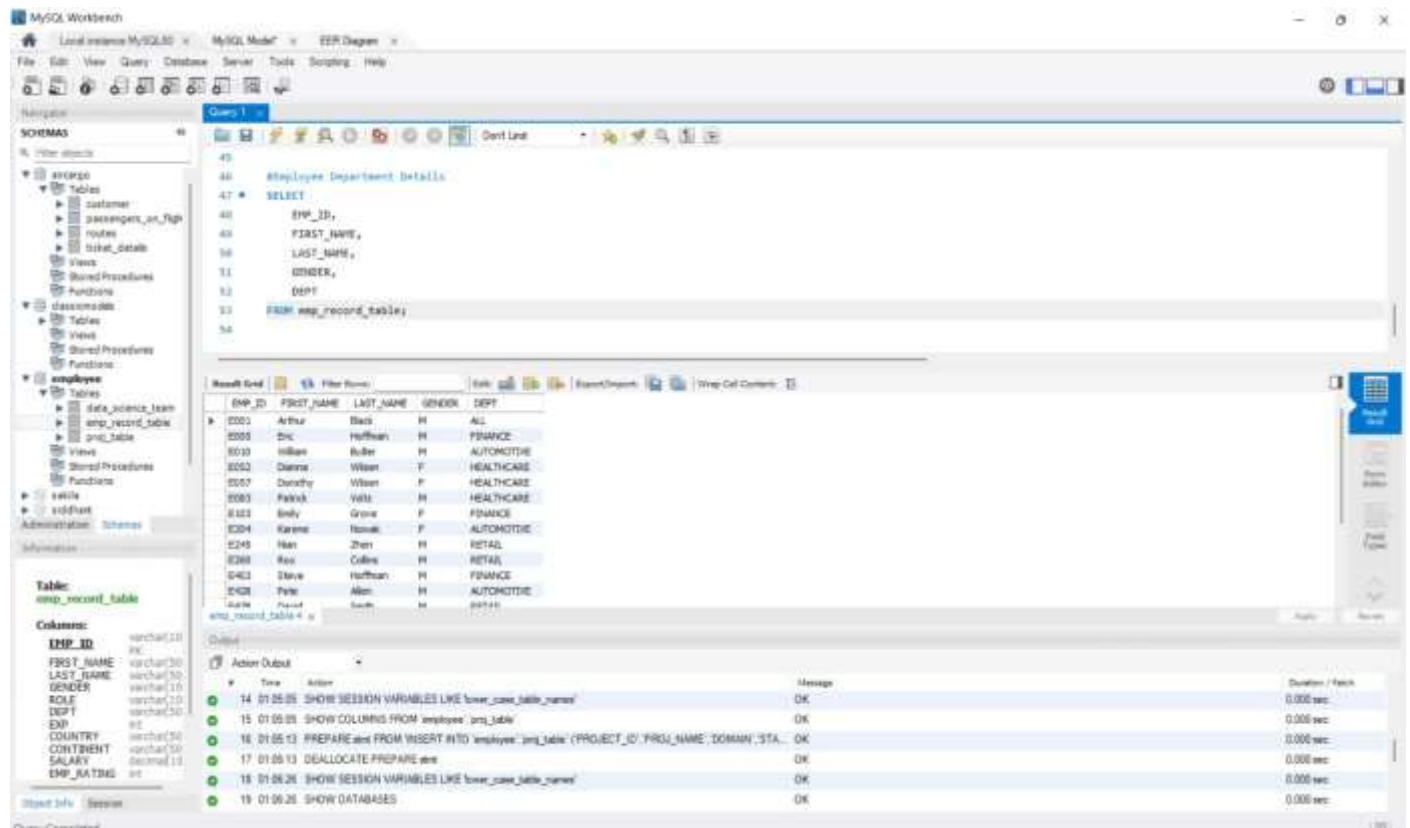
7.1 Employee Department Details

Problem Statement

Fetch EMP_ID, FIRST_NAME, LAST_NAME, GENDER, and DEPT from the employee record table to display employee and department details.

SQL Query

```
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    GENDER,
    DEPT
FROM emp_record_table;
```



The screenshot displays the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'employee' database selected. The 'Query Editor' window contains the following SQL query:

```
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    GENDER,
    DEPT
FROM emp_record_table;
```

The 'Results' pane shows the output of the query, which is a table with 5 columns: EMP_ID, FIRST_NAME, LAST_NAME, GENDER, and DEPT. The table contains 10 rows of data.

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	DEPT
0001	Arthur	Baer	M	ALL
0002	Eric	Hoffman	M	FINANCE
0003	William	Buller	M	AUTOMOTIVE
0004	Dianne	Wheat	F	HEALTHCARE
0005	Dorothy	Wheat	F	HEALTHCARE
0006	Patrick	Walt	M	HEALTHCARE
0007	Emily	Grove	F	FINANCE
0008	Karen	Rowe	F	AUTOMOTIVE
0009	Nan	Zhen	M	RETAIL
0010	Reo	Collins	M	RETAIL

The 'Action Output' pane at the bottom shows the execution progress of the query, including the time taken and the number of rows affected.

Explanation

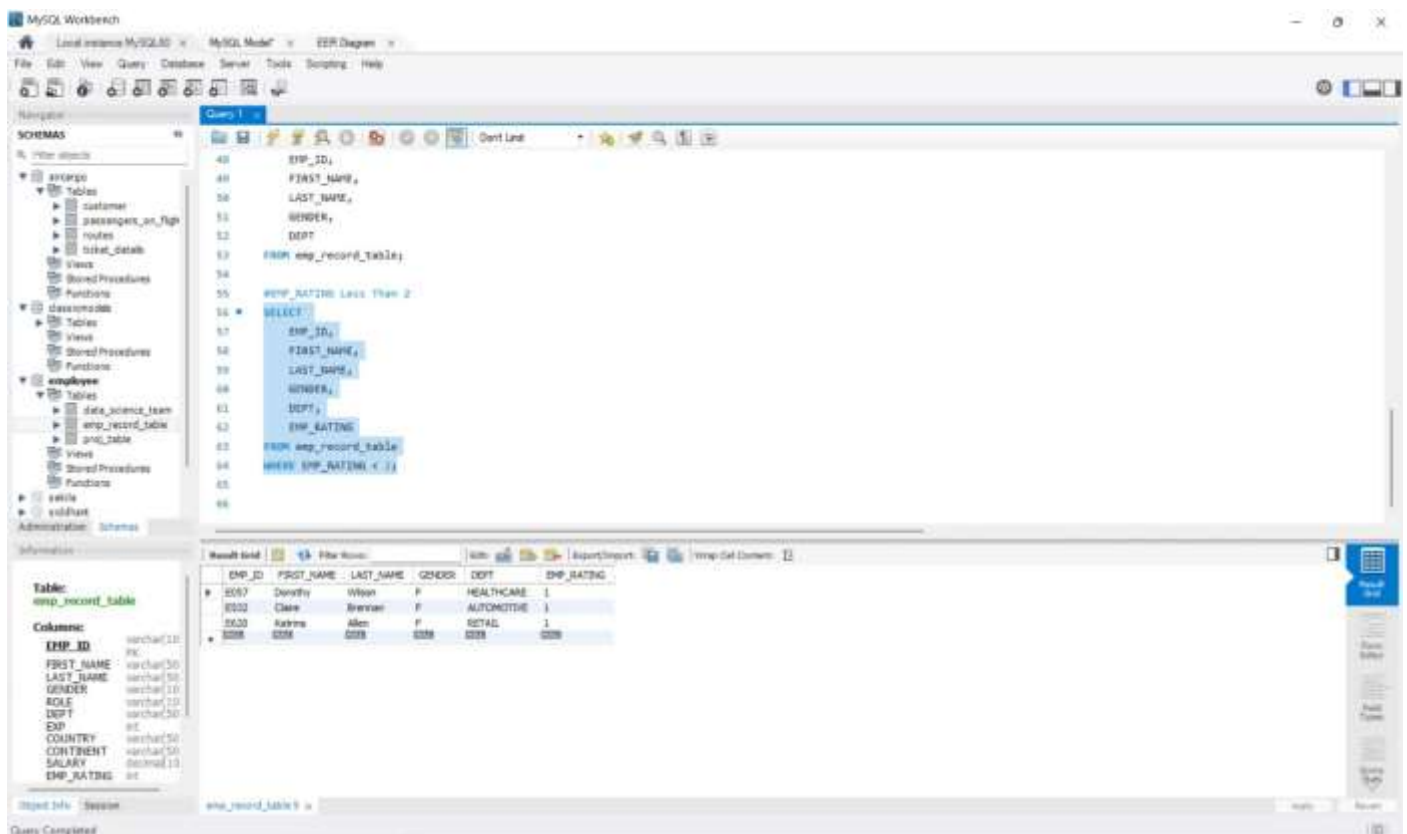
This query retrieves basic employee information along with their department from the emp_record_table. It helps the HR department understand employee distribution across various departments.

7.2 Employee Rating Analysis

Employees with EMP_RATING Greater Than 4

SQL Query

```
SELECT  
EMP_ID,  
FIRST_NAME,  
LAST_NAME,  
GENDER,  
DEPT,  
EMP_RATING  
FROM emp_record_table  
WHERE EMP_RATING > 4;
```



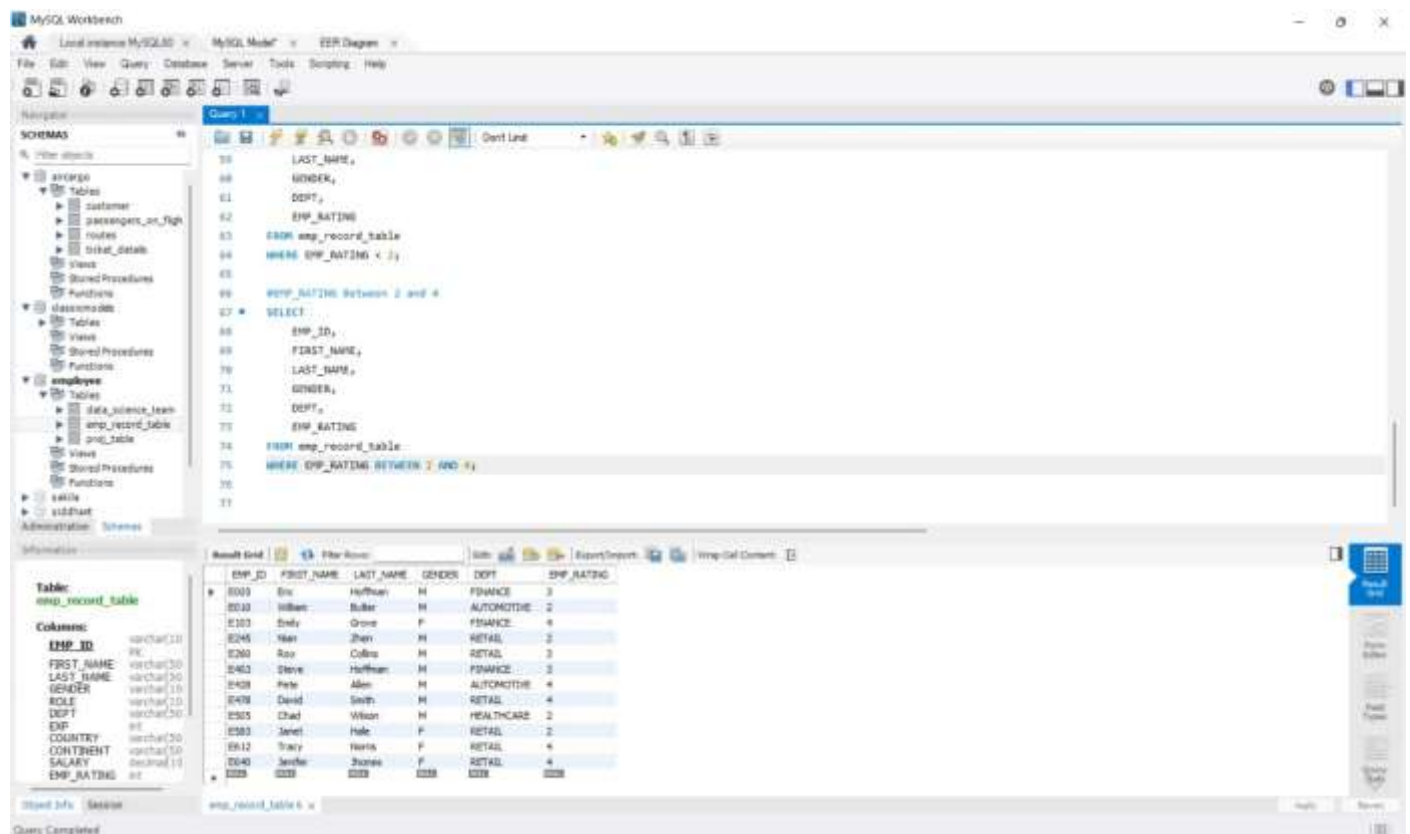
Explanation

This query retrieves employees who have excellent performance ratings greater than 4. These employees are considered top performers in the organization.

Employees with EMP_RATING Between 2 and 4

SQL Query

```
SELECT  
    EMP_ID,  
    FIRST_NAME,  
    LAST_NAME,  
    GENDER,  
    DEPT,  
    EMP_RATING  
FROM emp_record_table  
WHERE EMP_RATING BETWEEN 2 AND 4;
```



Explanation

This query displays employees whose ratings fall between 2 and 4, representing average to good performers within the organization.

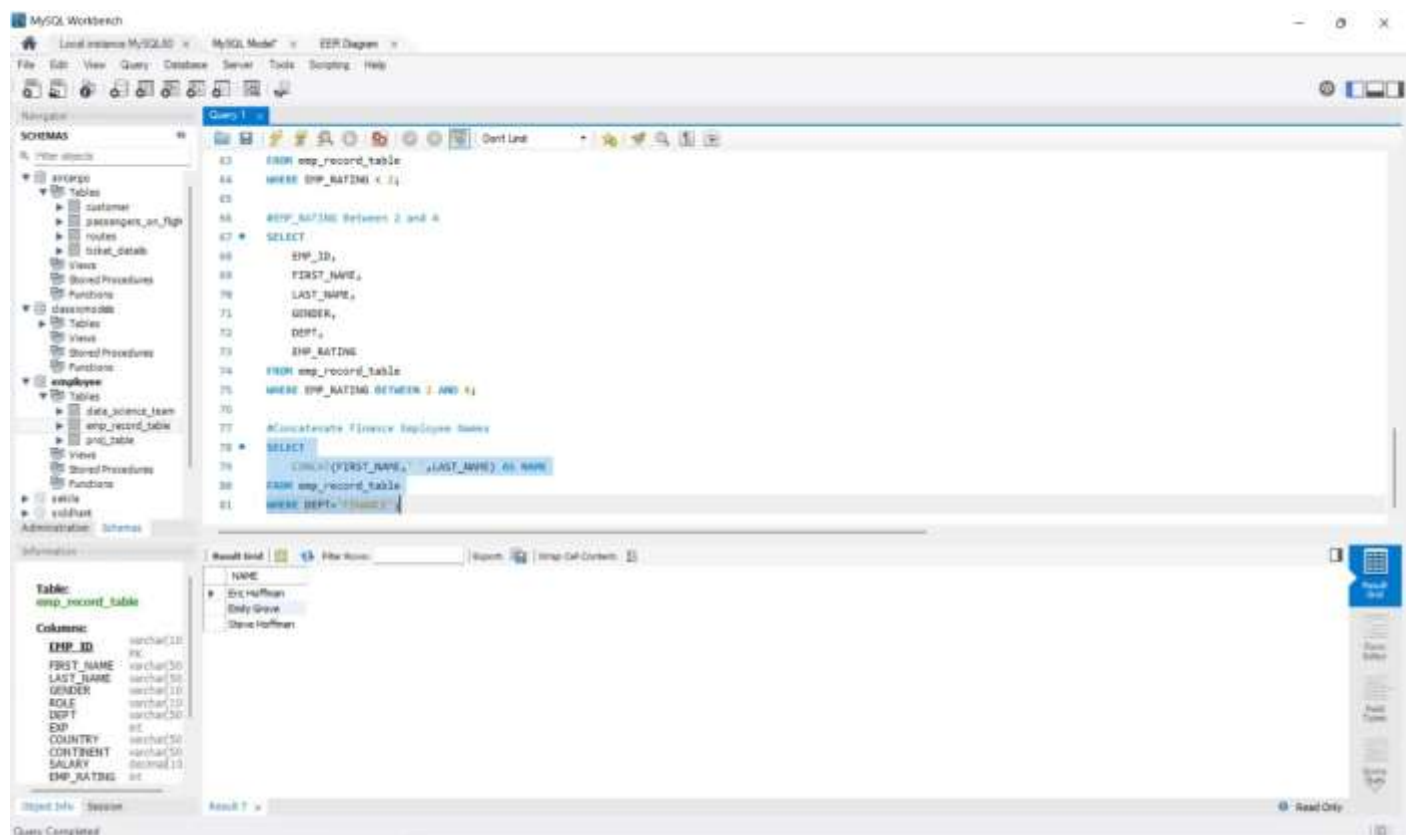
7.3 Finance Department Employee Names using CONCAT

Problem Statement

Concatenate FIRST_NAME and LAST_NAME of employees working in the Finance department.

SQL Query

```
SELECT  
    CONCAT(FIRST_NAME, ' ', LAST_NAME) AS NAME  
FROM emp_record_table  
WHERE DEPT='FINANCE';
```



Explanation

This query combines the first name and last name of employees from the Finance department into a single column named NAME using the CONCAT() function.

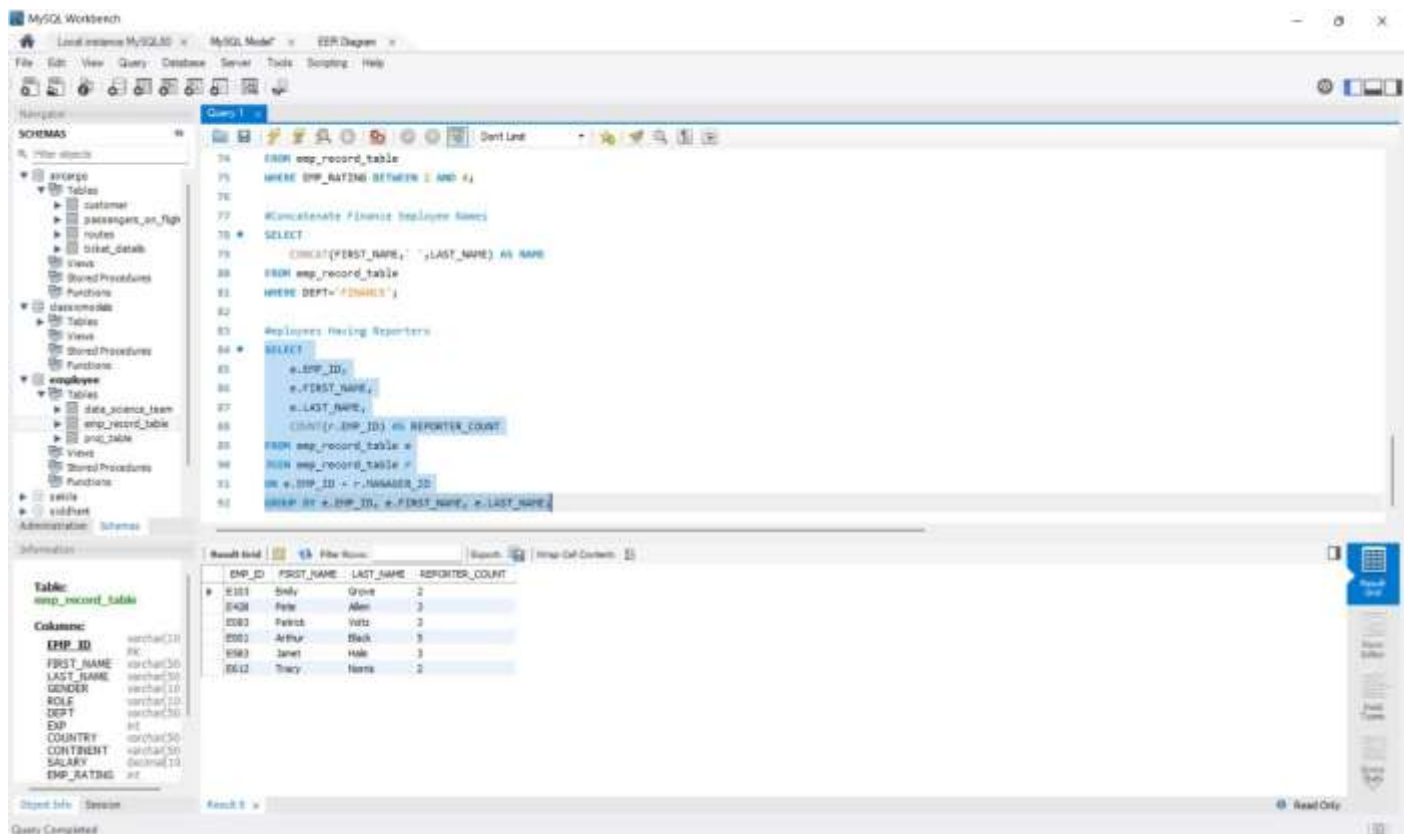
7.4 Employees Having Reporters

Problem Statement

List employees who have other employees reporting to them and display the number of reporters.

SQL Query

```
SELECT
    e.EMP_ID,
    e.FIRST_NAME,
    e.LAST_NAME,
    COUNT(r.EMP_ID) AS REPORTER_COUNT
FROM emp_record_table e
JOIN emp_record_table r
ON e.EMP_ID = r.MANAGER_ID
GROUP BY e.EMP_ID, e.FIRST_NAME, e.LAST_NAME;
```



The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists the 'employee' database. The 'Query' editor in the center contains the following SQL query:

```
SELECT
    e.EMP_ID,
    e.FIRST_NAME,
    e.LAST_NAME,
    COUNT(r.EMP_ID) AS REPORTER_COUNT
FROM emp_record_table e
JOIN emp_record_table r
ON e.EMP_ID = r.MANAGER_ID
GROUP BY e.EMP_ID, e.FIRST_NAME, e.LAST_NAME;
```

The 'Results' pane at the bottom displays the output of the query as a table with 4 columns: EMP_ID, FIRST_NAME, LAST_NAME, and REPORTER_COUNT. The table contains 6 rows of data.

EMP_ID	FIRST_NAME	LAST_NAME	REPORTER_COUNT
101	Scoty	Grove	2
102	Felix	Allen	2
103	Felicia	Vutts	2
104	Arthur	Black	3
105	Janet	Hall	3
106	Tracy	Harris	2

Explanation

This query uses a self join on the employee table to identify managers and count the number of employees reporting to each manager. It helps in understanding the reporting hierarchy within the organization.

7.5 Employees from Healthcare and Finance using UNION

Problem Statement

Display employees working in Healthcare and Finance departments using UNION.

SQL Query

```
SELECT *  
FROM emp_record_table  
WHERE DEPT='HEALTHCARE'
```

UNION

```
SELECT *  
FROM emp_record_table  
WHERE DEPT='FINANCE';
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
85  e.EMP_ID,  
86  e.FIRST_NAME,  
87  e.LAST_NAME,  
88  (CASE WHEN (r.EMP_ID) IS REPORTER_COUNT  
89  FROM emp_record_table e  
90  JOIN emp_record_table r  
91  ON e.EMP_ID = r.MANAGER_ID  
92  GROUP BY e.EMP_ID, e.FIRST_NAME, e.LAST_NAME)  
93  
94  #Healthcare and Finance Employees using UNION  
95  *  
96  SELECT *  
97  FROM emp_record_table  
98  WHERE DEPT='HEALTHCARE'  
99  
100 UNION  
101  
102 SELECT *  
103 FROM emp_record_table  
104 WHERE DEPT='FINANCE';
```

The Results grid displays the following data:

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	ROLE	DEPT	EMP_COUNTRY	CONTINENT	SALARY	EMP_RATING	MANAGER_ID	PROJ_ID
2002	Diana	Wheat	F	SENIOR DATA SCIENTIST	HEALTHCARE	4	CANADA	2300.00	3	2003	P103
2007	Dorothy	Wheat	F	SENIOR DATA SCIENTIST	HEALTHCARE	9	USA	7700.00	1	2003	P102
2003	Patrick	Vitti	M	MANAGER	HEALTHCARE	15	USA	6000.00	3	2001	2000
2005	Chad	Wheat	M	ASSOCIATE DATA SCIENTIST	HEALTHCARE	8	CANADA	3000.00	2	2003	P103
2009	Eric	Hoffman	M	LEAD DATA SCIENTIST	FINANCE	11	USA	8000.00	3	2103	P103
2103	Emily	Grant	F	MANAGER	FINANCE	14	CANADA	10900.00	4	2001	2000
2102	Steve	Hoffman	M	ASSOCIATE DATA SCIENTIST	FINANCE	4	USA	3000.00	2	2103	P103

The left sidebar shows the database schema with tables like emp_record_table and emp_record_table. The bottom status bar indicates 'Query Completed'.

Explanation

This query combines employees from the Healthcare and Finance departments into a single result set using the UNION operator.

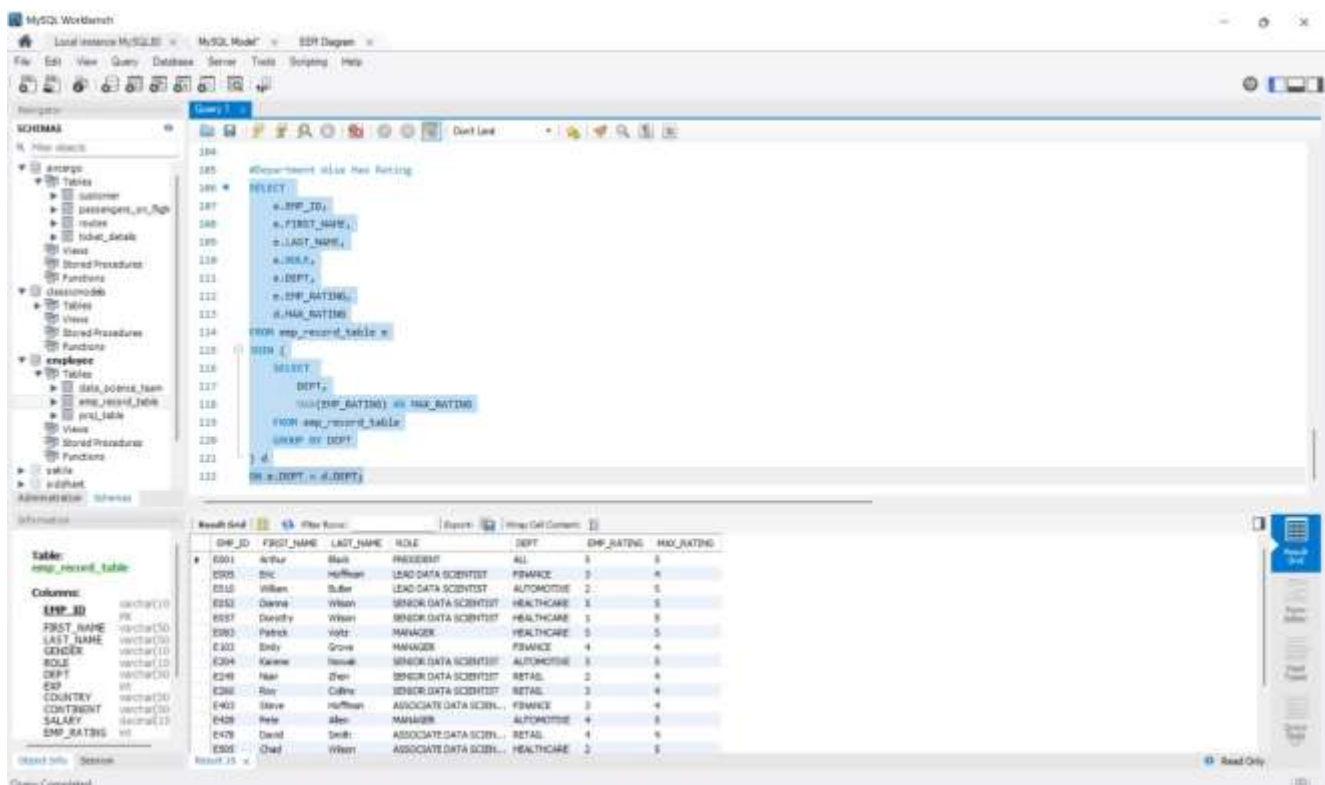
7.6 Department-wise Maximum Employee Rating

Problem Statement

Display employee details along with the maximum employee rating in each department.

SQL Query

```
SELECT
    e.EMP_ID,
    e.FIRST_NAME,
    e.LAST_NAME,
    e.ROLE,
    e.DEPT,
    e.EMP_RATING,
    d.MAX_RATING
FROM emp_record_table e
JOIN (
    SELECT
        DEPT,
        MAX(EMP_RATING) AS MAX_RATING
    FROM emp_record_table
    GROUP BY DEPT) d
ON e.DEPT = d.DEPT;
```



Explanation

This query calculates the minimum and maximum salary for employees grouped by their job roles. It helps analyze salary distribution across different positions in the organization.

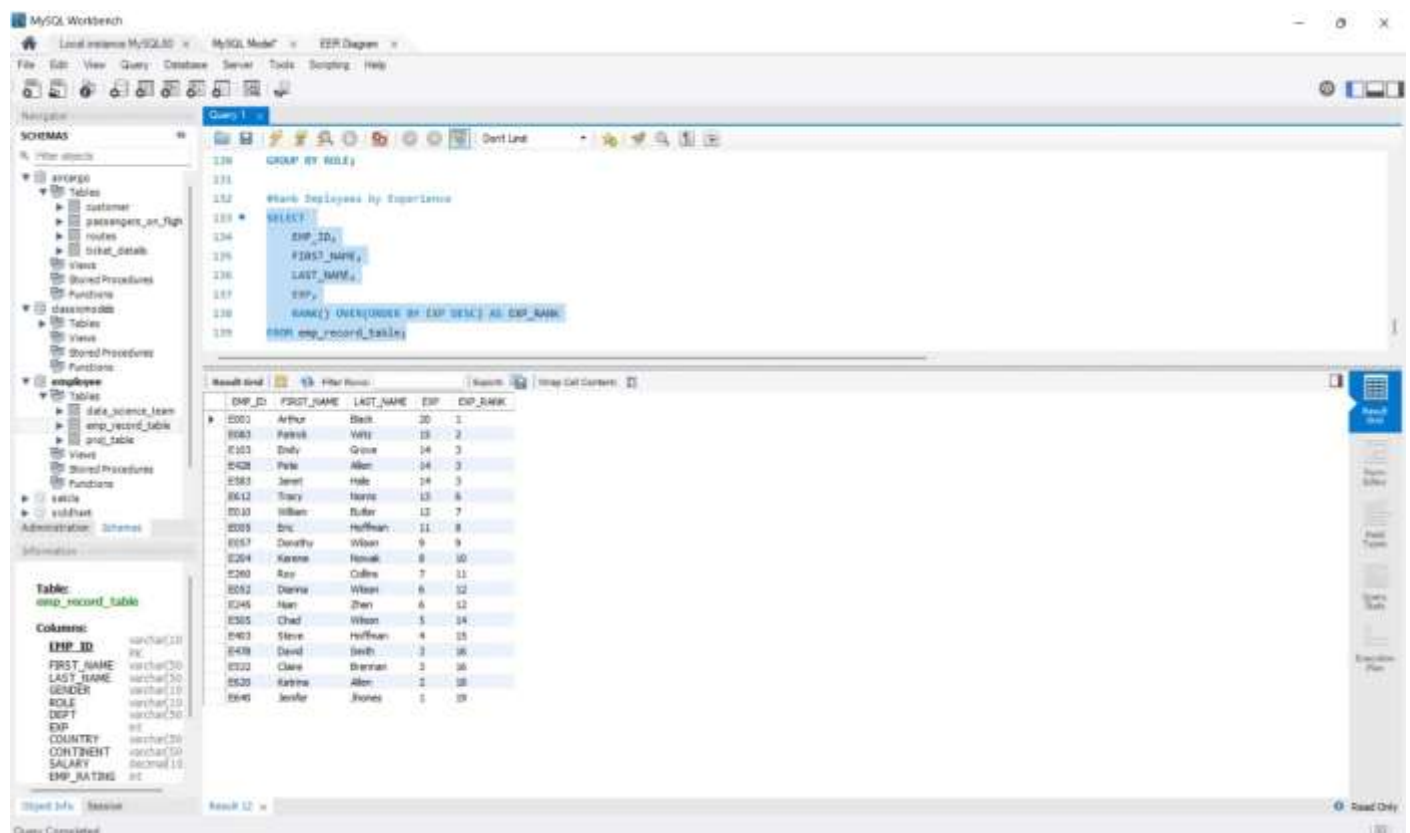
7.8 Employee Ranking based on Experience

Problem Statement

Assign ranks to employees based on their years of experience.

SQL Query

```
SELECT  
    EMP_ID,  
    FIRST_NAME,  
    LAST_NAME,  
    EXP,  
    RANK() OVER(ORDER BY EXP DESC) AS EXP_RANK  
FROM emp_record_table;
```



The screenshot shows the MySQL Workbench interface. The 'Query 1' window contains the SQL query to rank employees by experience. The 'Results' window displays the output of the query, showing columns EMP_ID, FIRST_NAME, LAST_NAME, EXP, and EXP_RANK. The results are sorted by experience in descending order.

EMP_ID	FIRST_NAME	LAST_NAME	EXP	EXP_RANK
1000	Arthur	Bach	30	1
1001	Patrick	Witt	18	2
1002	Dolly	Grave	14	3
1003	Pete	Allen	14	3
1004	James	Hale	14	3
1005	Tracy	Harris	13	6
1006	William	Butler	12	7
1007	Eric	Hoffman	11	8
1008	Dorothy	Wilson	9	9
1009	Karen	Rosak	8	10
1010	Roy	Collins	7	11
1011	Dianne	Wison	6	12
1012	Hari	Zhen	6	12
1013	Chad	Wilson	5	14
1014	Steve	Hoffman	4	15
1015	David	Smith	3	16
1016	Clare	Bennan	3	16
1017	Karina	Allen	2	18
1018	Jessie	Jones	1	19

Explanation

This query uses the RANK() window function to rank employees according to their experience in descending order. Employees with higher experience receive higher ranks.

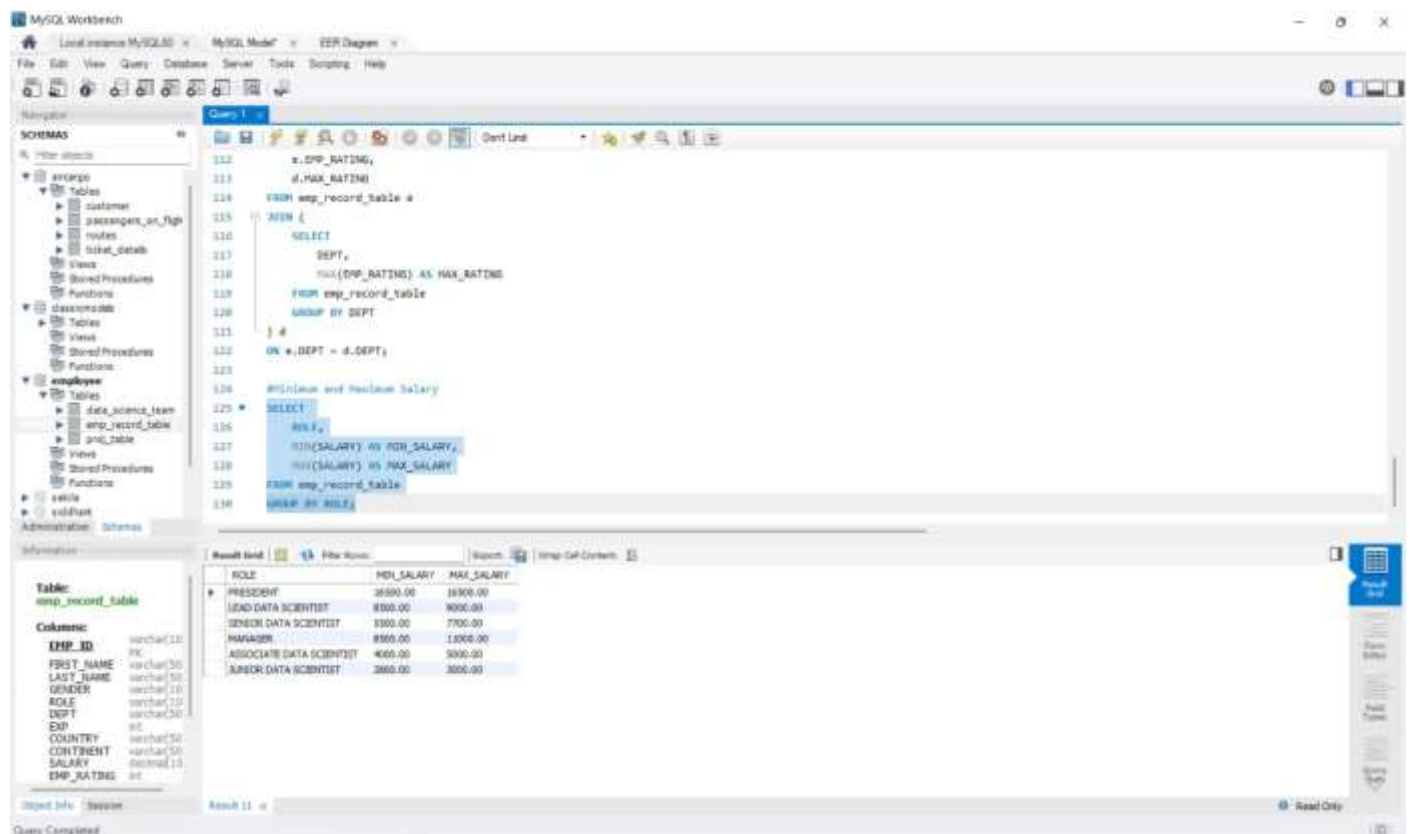
7.9 View Creation for High Salary Employees

Problem Statement

Create a view to display employees whose salary is greater than 6000.

SQL Query

```
CREATE VIEW high_salary_employees AS
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    COUNTRY,
    SALARY
FROM emp_record_table
WHERE SALARY > 6000;
```



Explanation

This query creates a virtual table (view) containing employees whose salary exceeds 6000. Views simplify data access and improve query reusability.

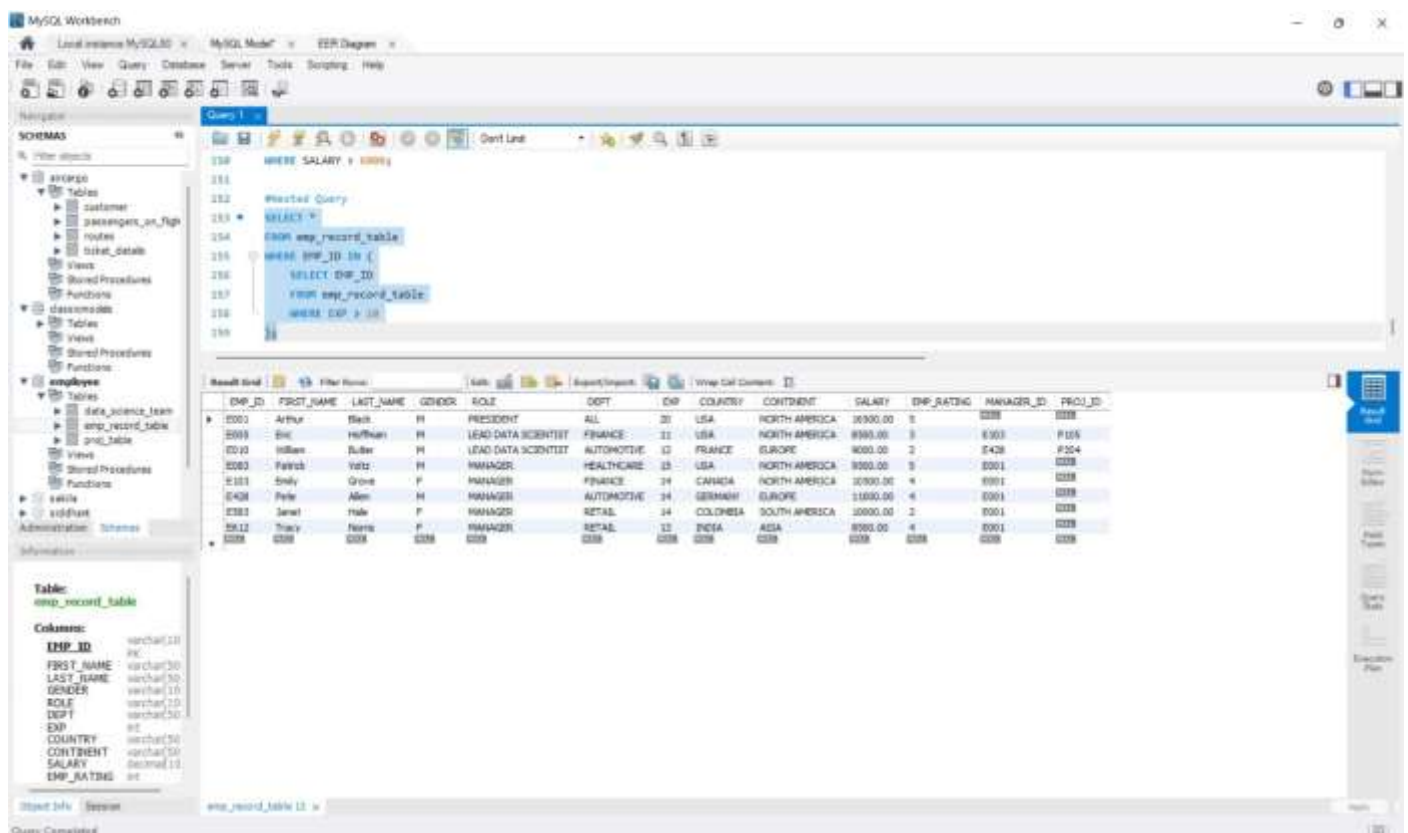
7.10 Nested Query for Experienced Employees

Problem Statement

Find employees with more than 10 years of experience using a nested query.

SQL Query

```
SELECT *  
FROM emp_record_table  
WHERE EMP_ID IN (  
    SELECT EMP_ID  
    FROM emp_record_table  
    WHERE EXP > 10  
);
```



The screenshot shows the MySQL Workbench interface. The 'Query' window displays a nested SQL query. The 'Result Grid' shows the output of the query, which lists employees with more than 10 years of experience. The 'Table: emp_record_table' window shows the schema of the table.

Query:

```
SELECT *  
FROM emp_record_table  
WHERE EMP_ID IN (  
    SELECT EMP_ID  
    FROM emp_record_table  
    WHERE EXP > 10  
);
```

Result Grid:

EMP_ID	FIRST_NAME	LAST_NAME	GENDER	ROLE	DEPT	EXP	COUNTRY	CONTINENT	SALARY	EMP_RATING	MANAGER_ID	PROJ_ID
1000	Arthur	Bell	M	PRESIDENT	ALL	20	USA	NORTH AMERICA	10000.00	5	0000	0000
1001	Elie	Hoffman	M	LEAD DATA SCIENTIST	FINANCE	12	USA	NORTH AMERICA	8500.00	3	0000	P105
1002	William	Buller	M	LEAD DATA SCIENTIST	AUTOMOTIVE	12	FRANCE	EUROPE	8000.00	2	0000	P204
1003	Fayaz	Yalta	M	MANAGER	HEALTHCARE	10	USA	NORTH AMERICA	9000.00	3	0001	0000
1004	Emily	Grone	F	MANAGER	FINANCE	14	CANADA	NORTH AMERICA	10000.00	4	0001	0000
1005	Heidi	Allen	M	MANAGER	AUTOMOTIVE	14	GERMANY	EUROPE	11000.00	4	0001	0000
1006	Samuel	Hale	M	MANAGER	RETAIL	14	COLUMBIA	SOUTH AMERICA	10000.00	2	0001	0000
1007	Tracy	Norm	F	MANAGER	RETAIL	12	INDIA	ASIA	8000.00	4	0001	0000
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Table: emp_record_table

Columns:

- EMP_ID: varchar(10)
- FIRST_NAME: varchar(30)
- LAST_NAME: varchar(50)
- GENDER: varchar(10)
- ROLE: varchar(50)
- DEPT: varchar(50)
- EXP: int
- COUNTRY: varchar(50)
- CONTINENT: varchar(50)
- SALARY: decimal(15)
- EMP_RATING: int

Explanation

This nested query retrieves employees whose experience exceeds 10 years. Subqueries help in filtering data based on conditions derived from another query.

7.11 Stored Procedure

Problem Statement

Create a stored procedure to retrieve employees with experience greater than 3 years.

SQL Query

DELIMITER //

CREATE PROCEDURE GetExperiencedEmployees()

BEGIN

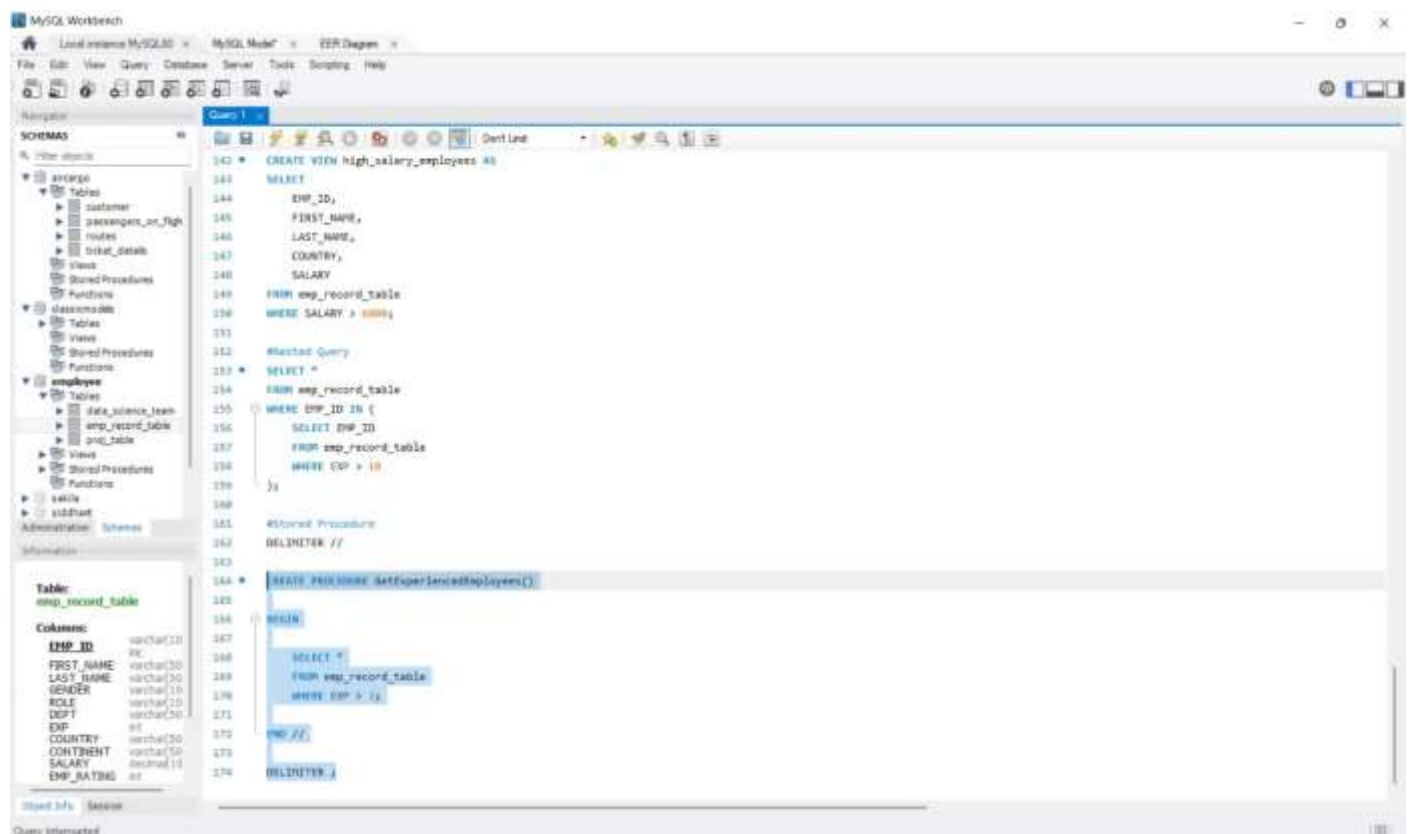
SELECT *

FROM emp_record_table

WHERE EXP > 3;

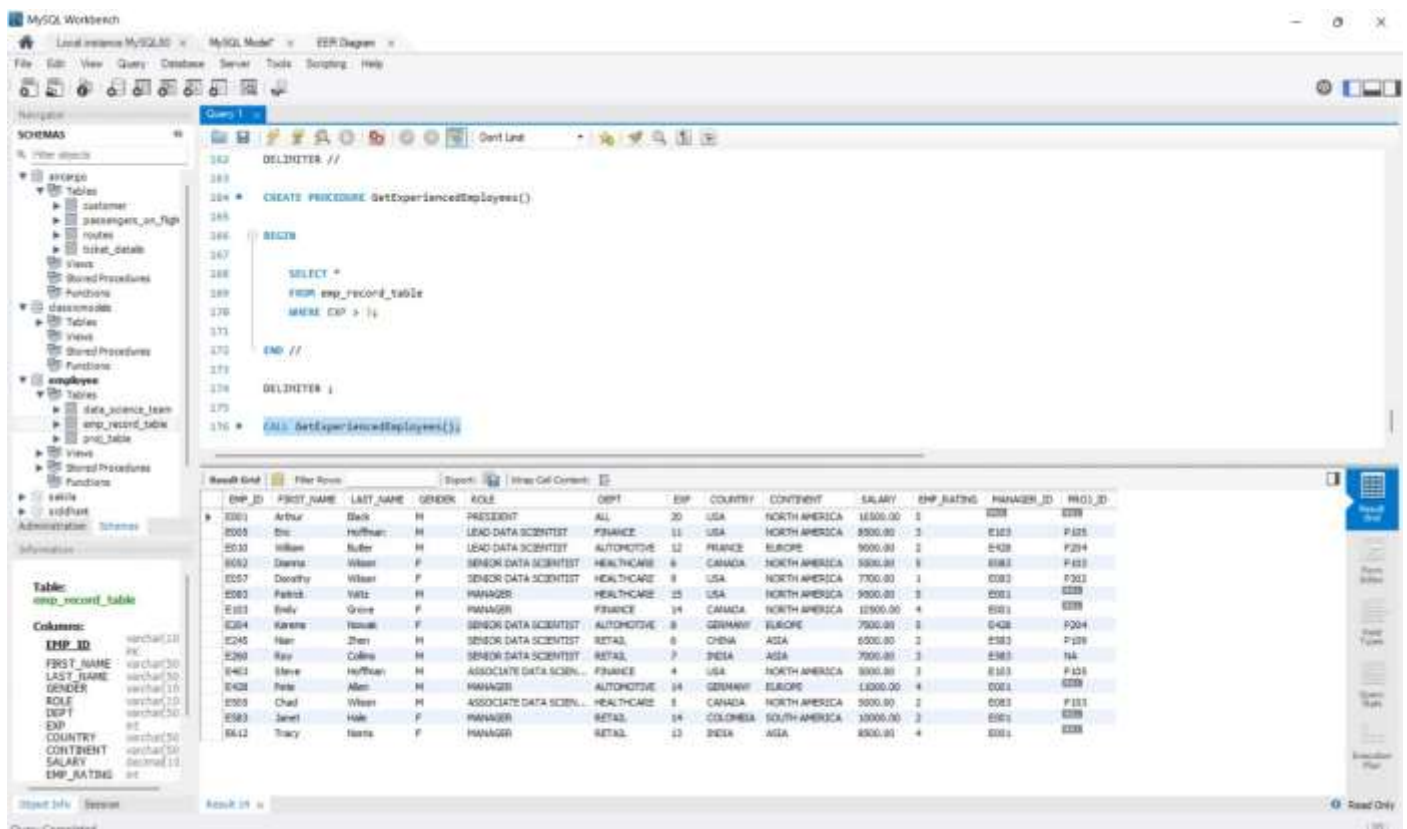
END //

DELIMITER ;



Execute Procedure

CALL GetExperiencedEmployees();



Explanation

This stored procedure retrieves employees having more than 3 years of experience. Stored procedures improve reusability and reduce repeated SQL coding.

7.12 Stored Function for Job Profile Validation

Problem Statement

Create a function to assign standard job profiles based on employee experience.

SQL Query

```
DELIMITER //
```

```
CREATE FUNCTION check_job_profile(exp_years INT)
```

```
RETURNS VARCHAR(100)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
DECLARE job_profile VARCHAR(100);
```

```
IF exp_years <= 2 THEN
```

```
    SET job_profile = 'JUNIOR DATA SCIENTIST';
```

```
ELSEIF exp_years > 2 AND exp_years <= 5 THEN
```

```
    SET job_profile = 'ASSOCIATE DATA SCIENTIST';
```

```
ELSEIF exp_years > 5 AND exp_years <= 10 THEN
```

```
    SET job_profile = 'SENIOR DATA SCIENTIST';
```

```
ELSEIF exp_years > 10 AND exp_years <= 12 THEN
```

```
    SET job_profile = 'LEAD DATA SCIENTIST';
```

```
ELSEIF exp_years > 12 AND exp_years <= 16 THEN
```

```
    SET job_profile = 'MANAGER';
```

```
ELSE
```

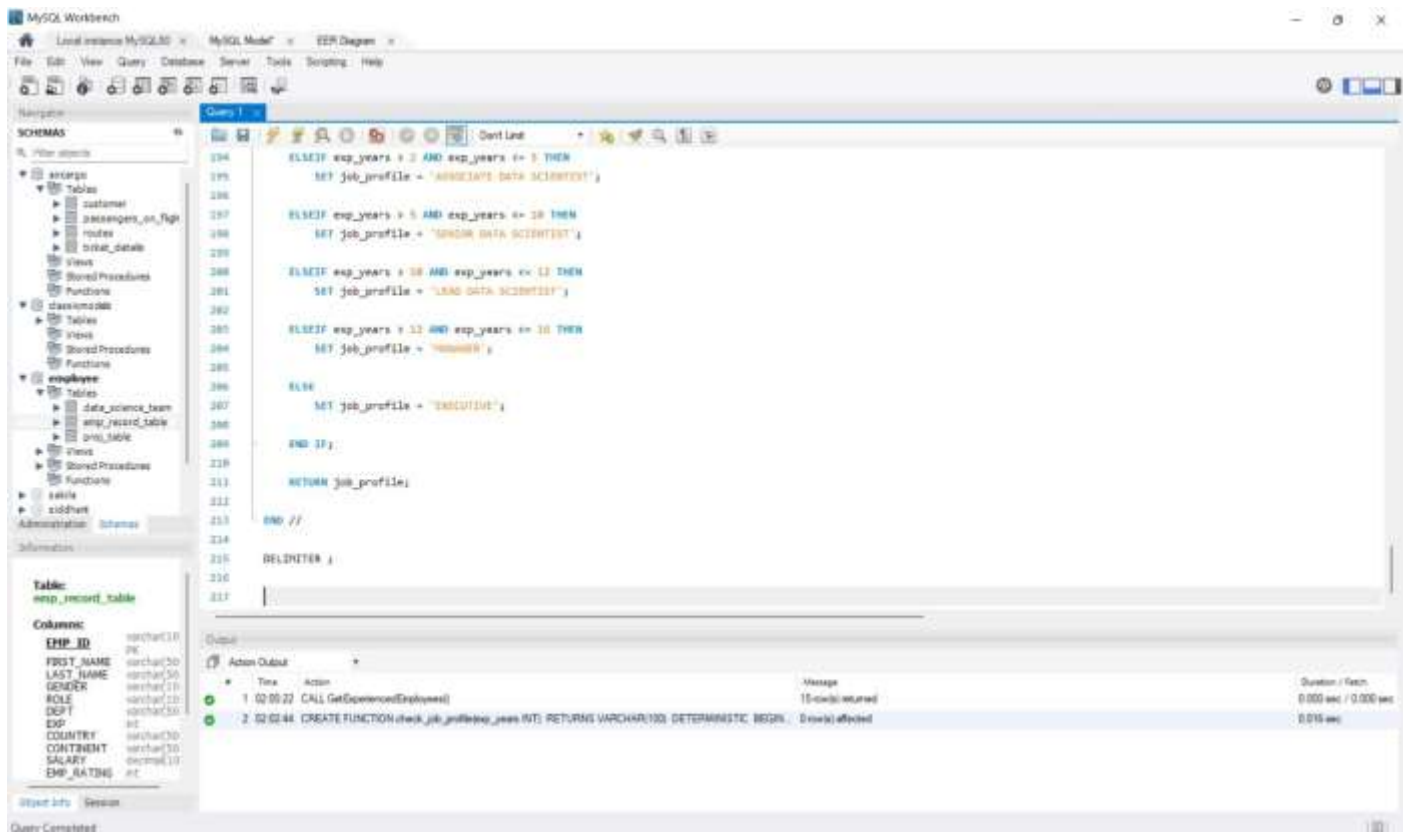
```
    SET job_profile = 'EXECUTIVE';
```

```
END IF;
```

```
RETURN job_profile;
```

```
END //
```

```
DELIMITER ;
```

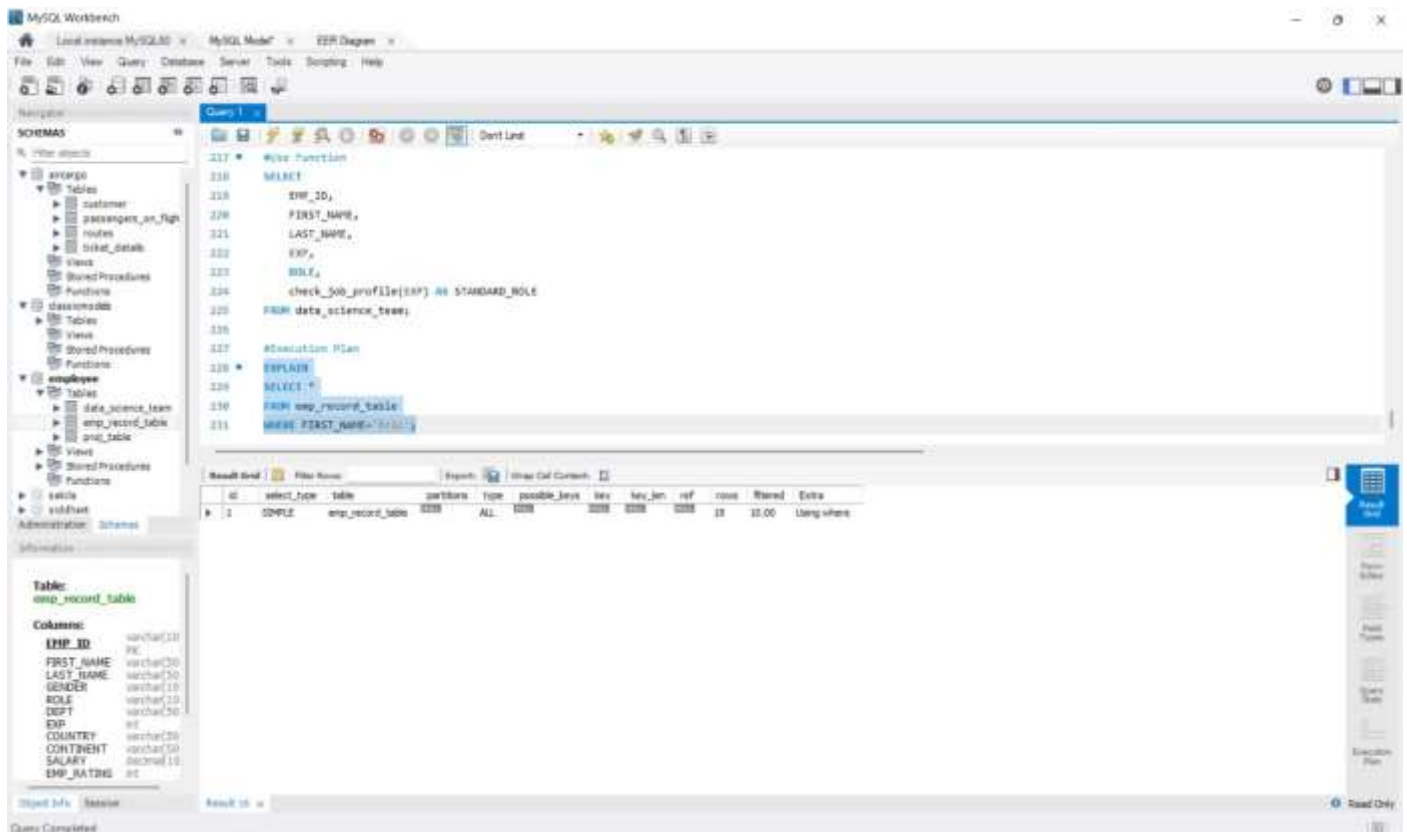


Function Execution

```

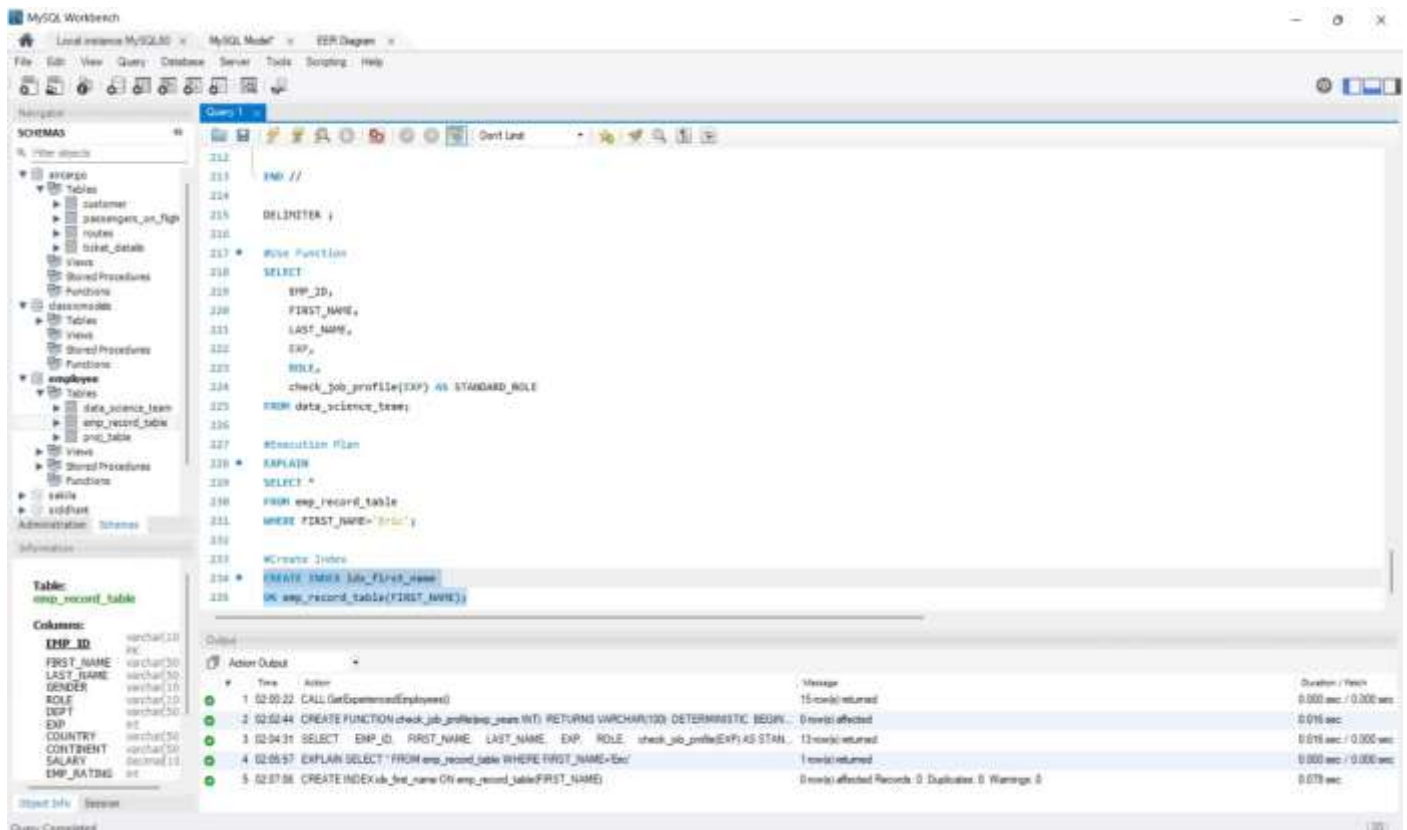
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    EXP,
    ROLE,
    check_job_profile(EXP) AS STANDARD_ROLE
FROM data_science_team;

```

Create Index

CREATE INDEX idx_first_name
ON emp_record_table(FIRST_NAME);



Execution Plan After Index

```
EXPLAIN
SELECT *
FROM emp_record_table
WHERE FIRST_NAME='Eric';
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'employee' database selected. The main editor shows a SQL query with line numbers 227 to 241. The query is an EXPLAIN statement for a SELECT query on the emp_record_table, filtered by FIRST_NAME='Eric'. Below the query editor, the 'Result Grid' tab is active, showing the execution plan. The plan indicates a full table scan (type: SIMPLE) on emp_record_table, with a ref key used for the WHERE clause. The bottom left pane shows the table structure for emp_record_table, listing columns like EMP_ID, FIRST_NAME, LAST_NAME, GENDER, ROLE, DEPT, EXP, COUNTRY, CONTINENT, SALARY, and EMP_RATING with their respective data types.

```
227 #Execution Plan
228 * EXPLAIN
229 SELECT *
230 FROM emp_record_table
231 WHERE FIRST_NAME='Eric';
232
233 #Create Index
234 * CREATE INDEX idx_first_name
235 ON emp_record_table(FIRST_NAME);
236
237 #Check Execution Plan Again
238 * EXPLAIN
239 SELECT *
240 FROM emp_record_table
241 WHERE FIRST_NAME='Eric';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	Stored	Extra
1	SIMPLE	emp_record_table		ref	idx_first_name	idx_first_name	203	const	1	300.00	SELECT

Table: emp_record_table

Columns:

- EMP_ID: int
- FIRST_NAME: varchar(50)
- LAST_NAME: varchar(50)
- GENDER: varchar(10)
- ROLE: varchar(10)
- DEPT: varchar(50)
- EXP: int
- COUNTRY: varchar(50)
- CONTINENT: varchar(50)
- SALARY: decimal(10,2)
- EMP_RATING: int

Explanation

Indexes improve database query performance by reducing the time required to search records. This query demonstrates query optimization using indexing.

7.14 Bonus Calculation

Problem Statement

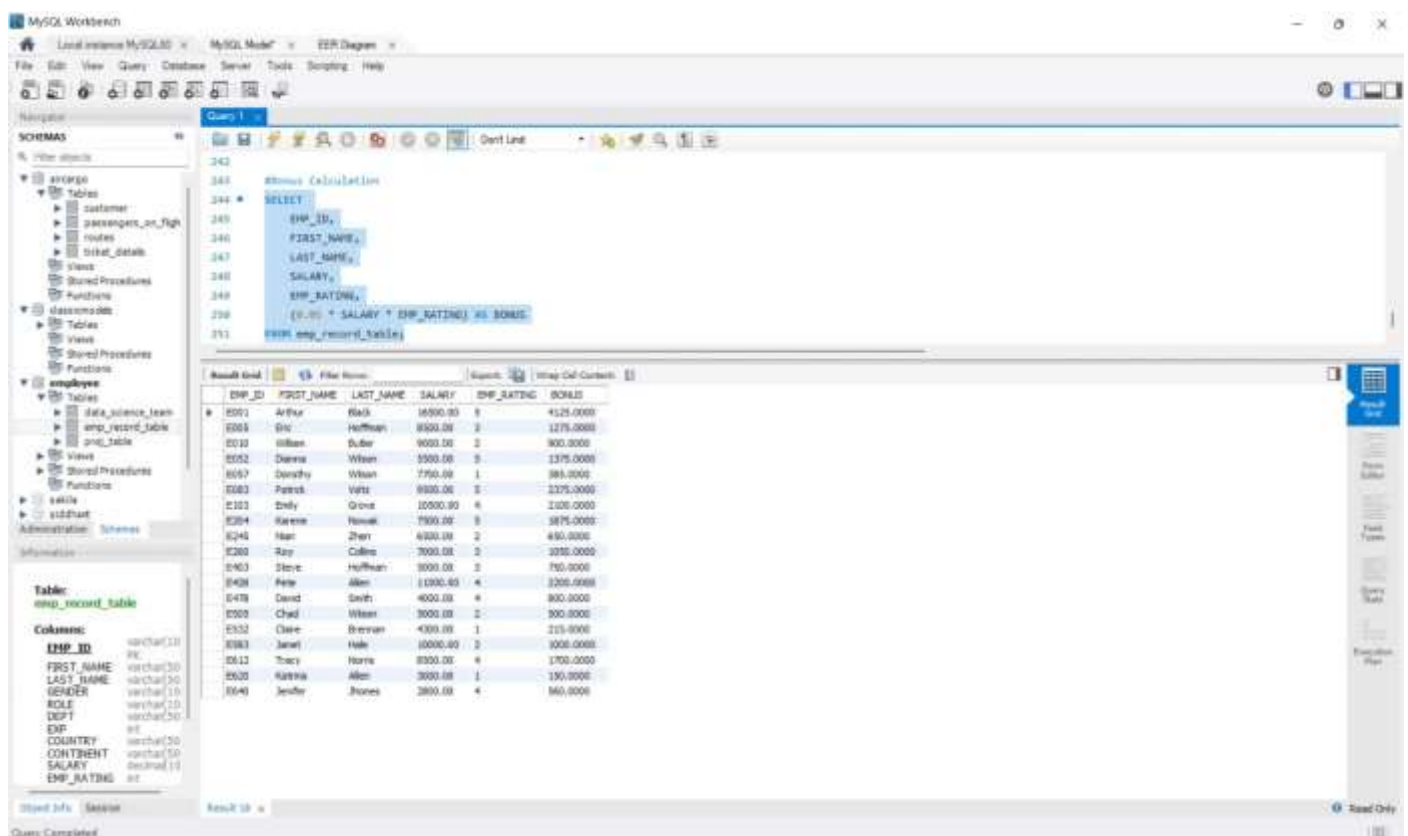
Calculate bonus for employees based on salary and employee rating.

Formula:

$5\% \text{ of Salary} \times \text{EMP_RATING}$

SQL Query

```
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    SALARY,
    EMP_RATING,
    (0.05 * SALARY * EMP_RATING) AS BONUS
FROM emp_record_table;
```



The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the SQL query for bonus calculation. The 'Results' tab shows the output of the query, which includes columns for EMP_ID, FIRST_NAME, LAST_NAME, SALARY, EMP_RATING, and BONUS. The results are sorted by EMP_ID in ascending order.

EMP_ID	FIRST_NAME	LAST_NAME	SALARY	EMP_RATING	BONUS
1001	Arba	Rick	16500.00	3	4125.0000
1005	Shir	Hoffman	8500.00	2	1275.0000
1010	William	Butler	9000.00	2	900.0000
1052	Danna	Wilson	5500.00	3	1375.0000
1057	Dorothy	Wilson	7750.00	1	386.8750
1063	Patsky	Vatta	8800.00	3	2375.0000
1103	Emily	Gross	10500.00	4	2100.0000
1204	Karen	Rowat	7900.00	3	1875.0000
1244	Han	Zhen	6300.00	2	630.0000
1260	Roy	Collins	7000.00	2	1050.0000
1403	Steve	Hoffman	3000.00	2	750.0000
1408	Pete	Allen	11000.00	4	2200.0000
1478	David	Smith	4000.00	4	800.0000
1500	Chad	Wilson	3000.00	2	300.0000
1552	Clare	Brown	4000.00	1	200.0000
1583	Janet	Hale	10000.00	2	1000.0000
1612	Tracy	Harris	8500.00	4	1700.0000
1620	Kathia	Allen	3000.00	1	150.0000
1640	Jeffrey	Jones	2800.00	4	560.0000

Explanation

This query calculates employee bonuses based on their salary and performance rating. Employees with higher ratings receive higher bonuses.

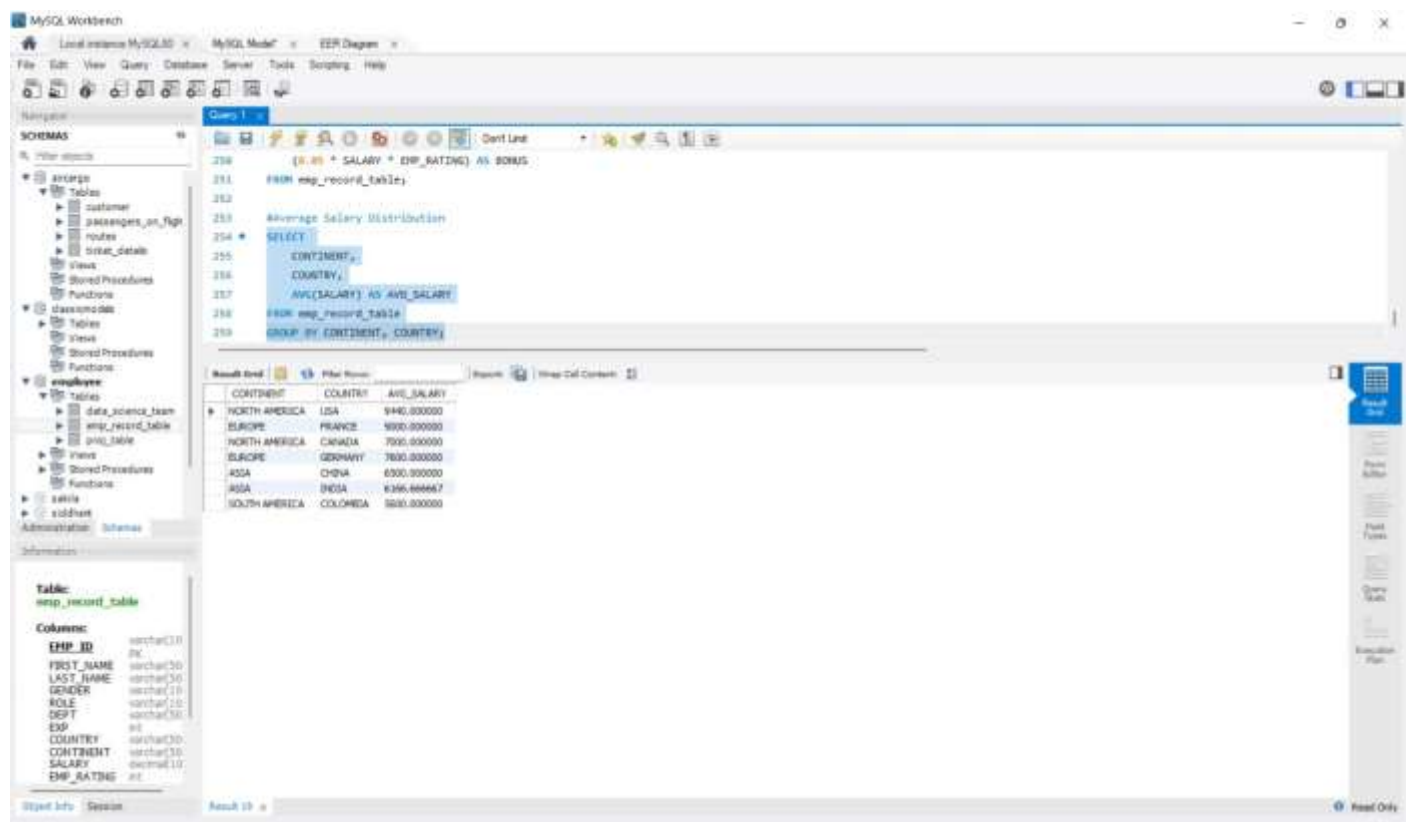
7.15 Average Salary Distribution

Problem Statement

Calculate average salary distribution based on continent and country.

SQL Query

```
SELECT  
    CONTINENT,  
    COUNTRY,  
    AVG(SALARY) AS AVG_SALARY  
FROM emp_record_table  
GROUP BY CONTINENT, COUNTRY;
```



The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'employee' database selected. The 'Tables' list includes 'emp_record_table'. The 'Query' pane shows the following SQL query:

```
SELECT  
    CONTINENT,  
    COUNTRY,  
    AVG(SALARY) AS AVG_SALARY  
FROM emp_record_table  
GROUP BY CONTINENT, COUNTRY;
```

The 'Results' pane shows the output of the query, which is a table with three columns: CONTINENT, COUNTRY, and AVG_SALARY. The data is as follows:

CONTINENT	COUNTRY	AVG_SALARY
NORTH AMERICA	USA	940.000000
EUROPE	FRANCE	900.000000
NORTH AMERICA	CANADA	700.000000
EUROPE	GERMANY	700.000000
ASIA	CHINA	600.000000
ASIA	INDIA	636.666667
SOUTH AMERICA	COLOMBIA	500.000000

The 'Table: emp_record_table' pane on the left shows the structure of the table, including columns like EMP_ID, FIRST_NAME, LAST_NAME, GENDER, ROLE, DEPT, EXP, COUNTRY, and CONTINENT.

Explanation

This query calculates the average salary for employees grouped by continent and country. It helps analyse salary trends across geographical regions.

8. Conclusion

The Employee Performance Mapping project successfully demonstrated the implementation of SQL concepts for database management and employee performance analysis. Various SQL operations such as joins, aggregate functions, subqueries, views, stored procedures, functions, and indexing were used to analyse employee records effectively.

The project helped in understanding how SQL can be utilized in real-world business scenarios for performance evaluation, reporting hierarchy management, salary analysis, and decision-making processes within an organization.